

MarketPlace

High-Level Design (HLD)

Platform Architecture — Version 0.2

<i>Company</i>	<i>Tiameds</i>
<i>Project / System</i>	<i>MarketPlace</i>
<i>Document Title</i>	<i>MarketPlace — High-Level Design (Platform Architecture, v0.2)</i>
<i>Author</i>	<i>Somil Merugawar</i>
<i>Approver</i>	<i>pending</i>
<i>Status</i>	<i>Draft</i>
<i>Last Updated</i>	<i>2026-09-07</i>

Table of Contents

Table of Contents	2
1. Document Overview	7
2. Objectives	7
3. Scope.....	7
4. Assumptions	8
5. Architecture Overview	8
6. System Context	9
7. High-Level Architecture	10
8. Component Architecture	11
9. Data Architecture.....	12
9.1 Seller onboarding → catalog listing.....	13
9.2 Quotation → order → billing → notification.....	14
10. Integration Architecture	15
11. Deployment Architecture	17
12. Security Architecture.....	18
13. Scalability & Availability	19
14. Observability & Monitoring	20
15. Disaster Recovery / Backup.....	20
16. CI/CD Architecture	21
17. Non-Functional Requirements	22
18. Risks & Constraints	22
19. Future Scalability	24
20. Appendix	24
Appendix A — Architecture Decision Records.....	25
Appendix B — AWS Services Reference.....	30
Compute	30
Database & caching.....	30
App integration & messaging.....	31
Platform & management.....	32
Security & identity	33
Networking & client-facing edge.....	33
Developer tools	33
Appendix C — Detailed Service Design Documents	35

Seller User Service — schema consolidation and deployment topology.....	35
01. Overview	35
02. Data ownership.....	35
03. Communication.....	37
Sync Inbound eligibility lookups	37
Platform Boot-time platform calls	37
04. Architecture diagram	37
05. Deployment.....	38
06. Scaling & region topology	38
07. Decisions worth writing down	39
Buyer User Service — schema, deployment, and auth-pattern alignment.....	40
01. Overview	40
02. Data ownership.....	40
03. Communication.....	41
Sync Inbound eligibility/profile lookups	41
Platform Boot-time platform calls	41
04. Architecture diagram	41
05. Deployment.....	42
06. Scaling & region topology	42
07. Decisions worth writing down	42
Order Management — the strict-consistency core of the order lifecycle.....	44
01. Overview	44
02. Data ownership.....	44
03. Communication.....	44
Sync IN Quotation Management → Order	44
Sync OUT Order → Product Management.....	44
Async OUT Order → Billing Management	45
Async OUT Order → Notification Service	45
04. Architecture diagram	45
05. Deployment.....	45
06. Scaling & region topology	46
07. Decisions worth writing down	46
Quotation Management — negotiation state and the single-table problem	47
01. Overview	47

02. Data ownership.....	47
03. Communication.....	47
Sync OUT Quotation → Product Management.....	47
Sync OUT Quotation → Order Management.....	47
04. Architecture diagram	48
05. Deployment.....	48
06. Scaling & region topology	49
07. Decisions worth writing down	49
Search Service — build-time vs. deployment-time infrastructure	50
01. Why this needs a build/deploy split at all.....	50
02. Build-time: what CI produces.....	50
Build Query-layer image	50
Build Index template & mapping.....	50
03. Deployment-time: what gets provisioned and reconciled	50
04. Terraform module layout	51
05. AWS service choices, and why	51
06. What a new region actually costs to spin up	52
07. Decisions worth writing down	52
Product Management — the catalog of record and the cost of splitting it out.....	53
01. Overview	53
02. Data ownership.....	53
03. Communication.....	54
Sync IN Order & Quotation Management.....	54
Async OUT Search Service.....	54
04. Architecture diagram	54
05. Deployment.....	54
06. Scaling & region topology	55
07. Decisions worth writing down	55
Billing Management — extracting invoicing without duplicating Order's data	56
01. Overview	56
02. Data ownership.....	56
03. Communication.....	56
Async IN Order Management → Billing.....	56
Sync OUT Billing → Order Management.....	56

Async OUT Billing → Notification Service	56
04. Architecture diagram	57
05. Deployment.....	57
06. Scaling & region topology	57
07. Decisions worth writing down	58
Notification Service — async-only by construction, not convention	59
01. Overview	59
02. Data ownership.....	59
03. Communication.....	59
Async IN Order Management.....	59
Async IN Billing Management.....	59
04. Architecture diagram	60
05. Deployment.....	60
06. Scaling & region topology	60
07. Decisions worth writing down	61
Appendix C (continued) — Platform Service Design Documents	62
API Gateway — the platform's sole client entry point.....	62
01. Overview	62
02. Data ownership.....	62
03. Communication.....	62
04. Architecture diagram	62
05. Deployment.....	63
06. Scaling & region topology	63
07. Decisions worth writing down	63
Config Service — externalized config, high availability over throughput.....	64
01. Overview	64
02. Data ownership.....	64
03. Communication.....	64
04. Architecture diagram	64
05. Deployment.....	65
06. Scaling & region topology	65
07. Decisions worth writing down	65
Discovery Service — the live registry that wires regions together	66
01. Overview	66

02. Data ownership.....	66
03. Communication.....	66
04. Architecture diagram	66
05. Deployment.....	66
06. Scaling & region topology	67
07. Decisions worth writing down	67

1. Document Overview

Field	Value
Document Title	MarketPlace — High-Level Design (Platform Architecture, v0.2)
Version	v0.2
Author	Somil Meugawar
Approver	pending
Created Date	Not recorded in source material (v0.1 baseline, undated)
Last Updated	2026-09-07
Status	Draft — underlying architecture decisions accepted (ADR-001 through ADR-009, Appendix A) document itself pending formal review sign-off

2. Objectives

- Decompose the existing monolith into 11 independently deployable services with clear, defensible ownership boundaries.
- Enable independent scaling per capability — User Service, Order, Quotation, Billing, and Search each have different, uncorrelated traffic shapes that a shared monolith cannot scale for separately.
- Support one-click multi-region deployment through parameterized infrastructure-as-code, rather than manual per-region setup (ADR-006).
- Establish clear per-team service ownership so capability boundaries map to organizational boundaries.
- Preserve data-ownership integrity during extraction — no service duplicates another service's data as a shortcut (ADR-004).

3. Scope

In scope: service boundaries and responsibilities for all 11 services. the AWS services each depends on inter-service communication (synchronous and event-driven) multi-region deployment topology security and trust boundaries as documented and migration-relevant risks specific to extracting each service from the monolith.

Out of scope: detailed API contracts/OpenAPI specifications CI/CD pipeline internals beyond the region-rollout mechanism non-platform application features frontend/client application design (client apps are treated as external consumers of the platform API).

Audience: backend and platform engineers implementing the split DevOps/SRE provisioning the AWS footprint security reviewers assessing trust boundaries engineering leadership sequencing the migration roadmap.

4. Assumptions

- **Assumption:** the backend already deploys via ECS/ECR with a working GitHub Actions pipeline (ADR-002, ADR-006) — this design builds on that tooling rather than replacing it.
- **Assumption:** compute, data, and CI/CD are already AWS-committed, giving an AWS-native bias throughout (ADR-002) self-hosted alternatives are named fallbacks only, not parallel plans.
- **Assumption:** the engineering team's existing background is Spring Boot services, which shaped several "obvious alternative" comparisons in the ADRs.
- **To Be Confirmed:** exact VPC CIDR ranges, subnet counts, and Availability Zone counts for compute and Aurora — the source Terraform references `var.vpc_id` and `var.private_subnet_ids` generically without concrete values.
- **To Be Confirmed:** WAF/DDoS posture, network ACL configuration, and audit-logging/monitoring strategy — none are specified in the source material.
- **To Be Confirmed:** compliance framework — none is named given the pharmaceutical-marketplace domain, data-privacy and drug-sale regulatory requirements are plausible candidates but unconfirmed.

5. Architecture Overview

- The platform is organized in three layers. A single **API Gateway** is the sole entry point for client traffic (ADR-008) — no domain service is directly reachable from the internet. Eight **domain services** own the business logic and data: Seller User, Buyer User, Order Management, Quotation Management, Product Management, Search Service, Billing Management, and Notification Service. Underneath, two **platform services** — Config Service and Discovery Service — provide configuration and service discovery to every other service, in every region (ADR-002).
- Service-to-service calls are synchronous only where the caller cannot return a correct answer without the result in the same request — an eligibility check, a live price/stock confirmation, a quote-acceptance handoff. Everything else runs asynchronously over Amazon EventBridge (ADR-003).
- Two deployment topology patterns coexist deliberately (ADR-007). Write-sensitive services — Order, Billing, Seller User, Buyer User, Quotation — run **single-writer with a warm standby region**, because Aurora PostgreSQL supports one writer region at a time and these domains cannot tolerate write conflicts. Read-heavy, lag-tolerant services — Product Management and Search — run **active-active across regions**, because a stale catalog read is a non-event but a lost order is not.

6. System Context

At the highest level, three external actor types interact with the platform — Buyers, Sellers, and back-office Admins — and the platform in turn depends on a small set of external systems for delivery and authentication. The existing monolith is shown as a source system still being decomposed, not a peer system.

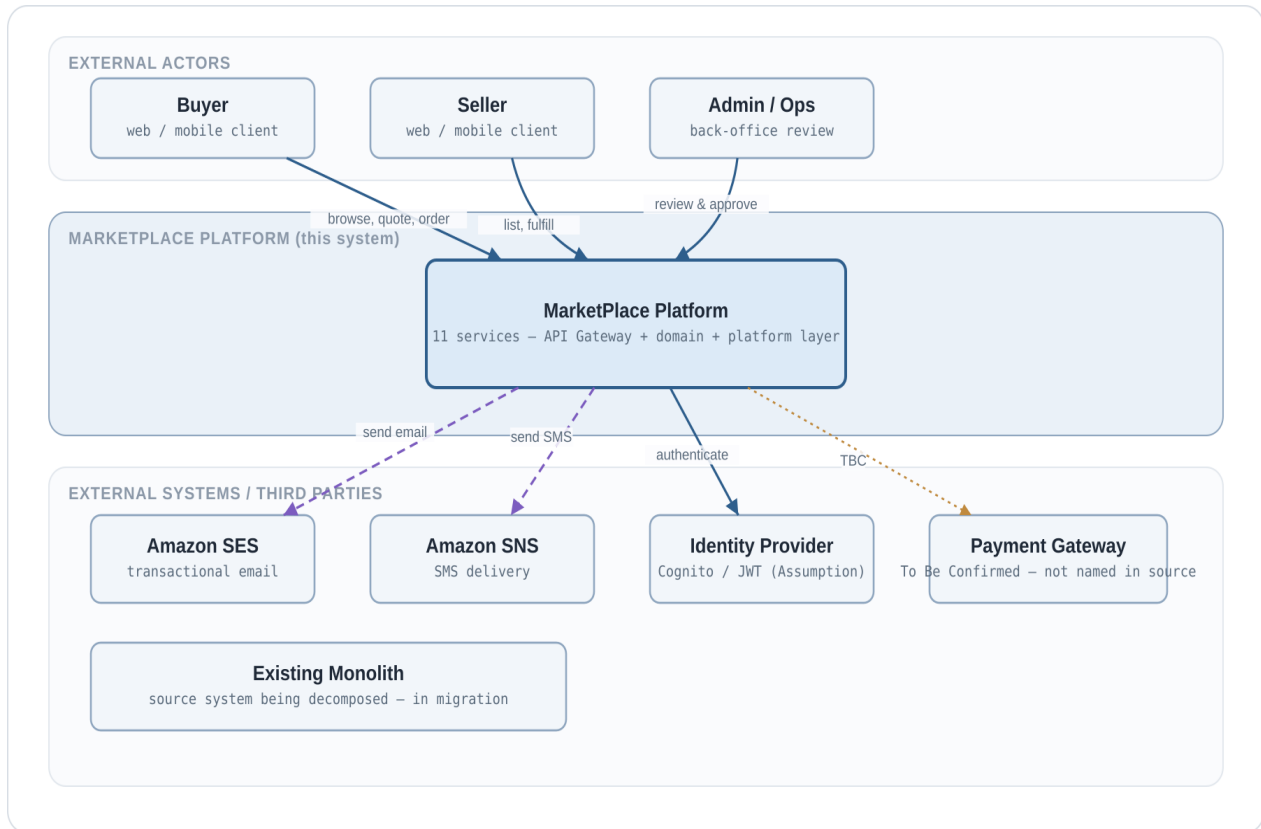
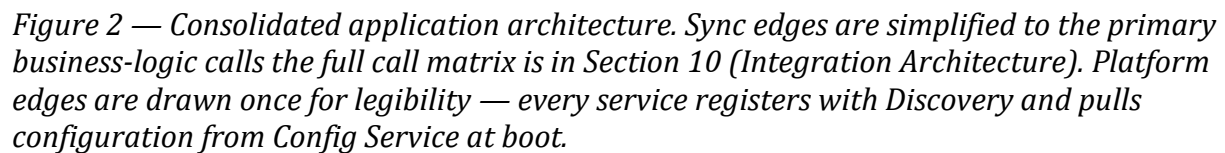


Figure 1 — System context. The Identity Provider is drawn as Cognito per the API Gateway design (Appendix C) but is marked Assumption since no explicit IdP decision record exists. The Payment Gateway is marked To Be Confirmed — no provider is named anywhere in the source material.

The diagram below consolidates all 11 services into one picture: solid navy arrows are synchronous calls, dashed purple arrows are asynchronous events, and dotted amber arrows represent the platform services every domain service depends on at boot.



8. Component Architecture

Component / Service	Responsibility	Owner / Team	Notes
Seller User	Seller identity end to end: profile, registration, onboarding documents, bank details, seller/coordinator auth	Identity & Accounts Team	Consolidates ~21 tables from 3 monolith packages into one schema single-writer with warm standby
Buyer User	Buyer identity: profile, registration, onboarding documents, buyer auth	Identity & Accounts Team	Same shape as Seller User, lighter volume (15 tables, 2 lifecycle stages)
Order Management	Orders, order lines, status transitions, returns/refunds	Order & Commerce Team	Strict-consistency core of the platform single-writer, never active-active (ADR-007)
Quotation Management	Buyer quote requests, seller responses, negotiation state	Order & Commerce Team	One table today (tbl_quote_request) backing two flows 3 cross-domain FKs to resolve via API post-split
Product Management	Catalog of record for every sellable item plus shared lookup masters	Catalog & Search Team	~62 tables, ~59% of the system's ~106 tables largest cross-cutting migration risk
Search Service	Query layer over an OpenSearch index rebuilt from catalog events	Catalog & Search Team	Owns no source data (ADR-005) index rebuild is the one Kafka (MSK) candidate edge
Billing Management	Invoices, payment records, ledger entries	Billing & Notifications Team	References Order by ID only, no line-item copy (ADR-004) physically separate DB from Order 14-day backup retention (vs. 7 elsewhere)
Notification Service	Email/SMS delivery for order and billing events	Billing & Notifications Team	Async-only by construction — exposes no REST endpoint at all
API Gateway	Sole client entry point: routing, authN/authZ, rate limiting	Platform Engineering Team	Managed Amazon API Gateway self-hosted Spring Cloud Gateway kept as fallback only (ADR-008)

Component / Service	Responsibility	Owner / Team	Notes
Config Service	Externalized, per-region configuration for every service	Platform Engineering Team	AppConfig + Parameter Store secrets explicitly excluded (Secrets Manager instead)
Discovery Service	Live service registry — who's up, in which region, at which address	Platform Engineering Team	AWS Cloud Map the actual mechanism behind one-click region rollout (ADR-002, ADR-006)

Full responsibilities, data ownership down to table level, deployment Terraform, and each service's own architecture diagram are in Appendix C.

9. Data Architecture

Each domain service owns its data exclusively — no shared database, no cross-service foreign keys once extraction completes (opaque, API-resolved references instead see Quotation Management, Appendix C). Master/reference data (state, district, document-type masters, etc.) remains a shared read-only module rather than a numbered service (ADR-001).

Service	Primary store	Approx. tables/entities	Consistency model
Seller User	Aurora PostgreSQL	~21 (consolidating 3 packages)	Single-writer, strong
Buyer User	Aurora PostgreSQL	~15	Single-writer, strong
Order Management	Aurora PostgreSQL	8 (order, lines, status history, returns, refunds)	Single-writer, strong
Quotation Management	Aurora PostgreSQL	1 today (tbl_quote_request)	Single-writer, strong
Product Management	Aurora PostgreSQL (Global Database) + ElastiCache	~62 (~59% of system total)	Active-active reads, eventually consistent globally
Search Service	OpenSearch (no source-of-truth store)	N/A — rebuilt index only	Eventually consistent (ADR-005)

Service	Primary store	Approx. tables/entities	Consistency model
Billing Management	Aurora PostgreSQL (physically separate from Order)	2 extracted + ledger_entries (new)	Single-writer, strong
Notification Service	None (optional delivery log only)	0	N/A

Data flows. Two representative end-to-end flows illustrate how data moves through the system in practice:

9.1 Seller onboarding → catalog listing

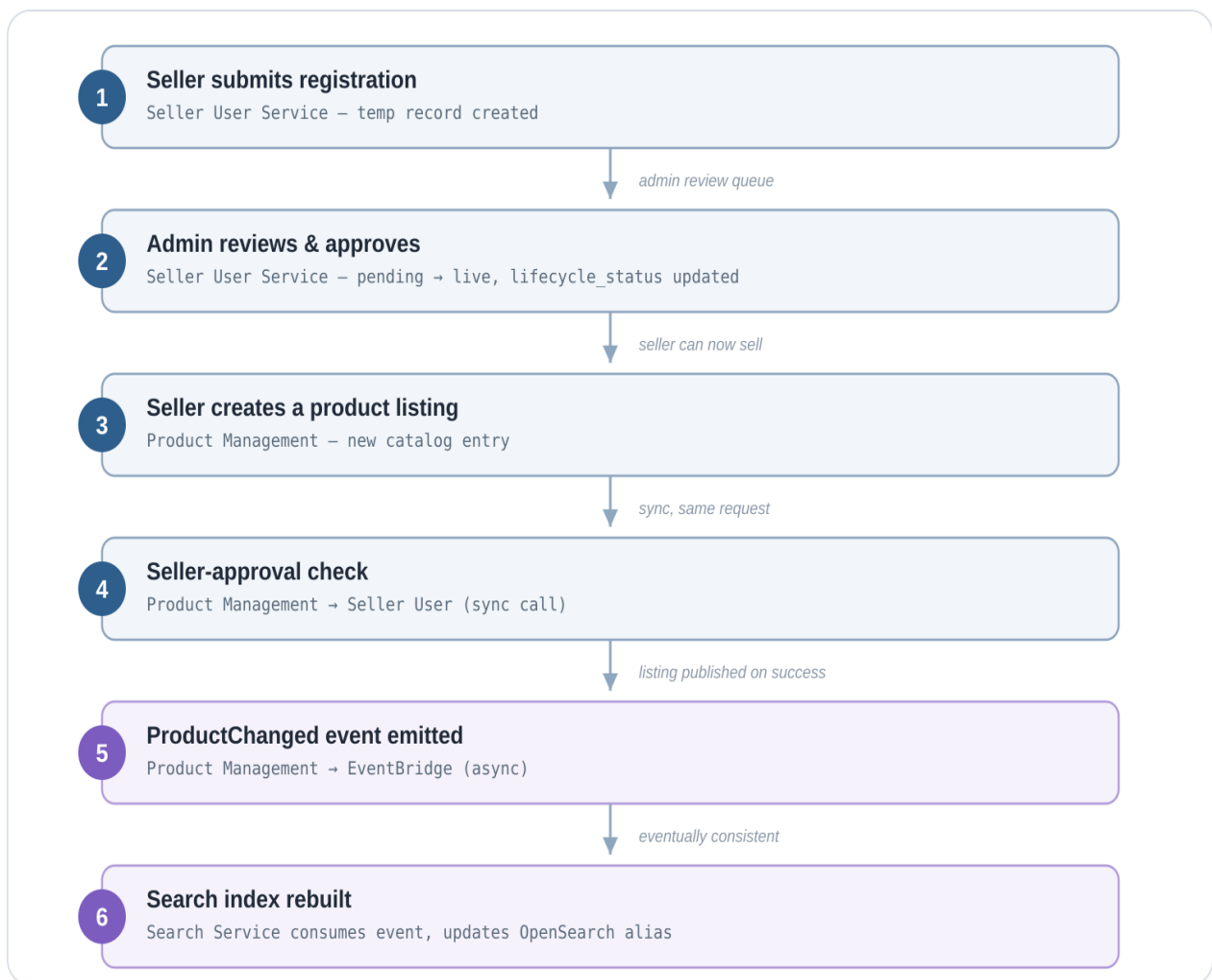


Figure 3 — Steps 1–4 are synchronous the index rebuild (5–6) is asynchronous and eventually consistent by design (ADR-005).

9.2 Quotation → order → billing → notification

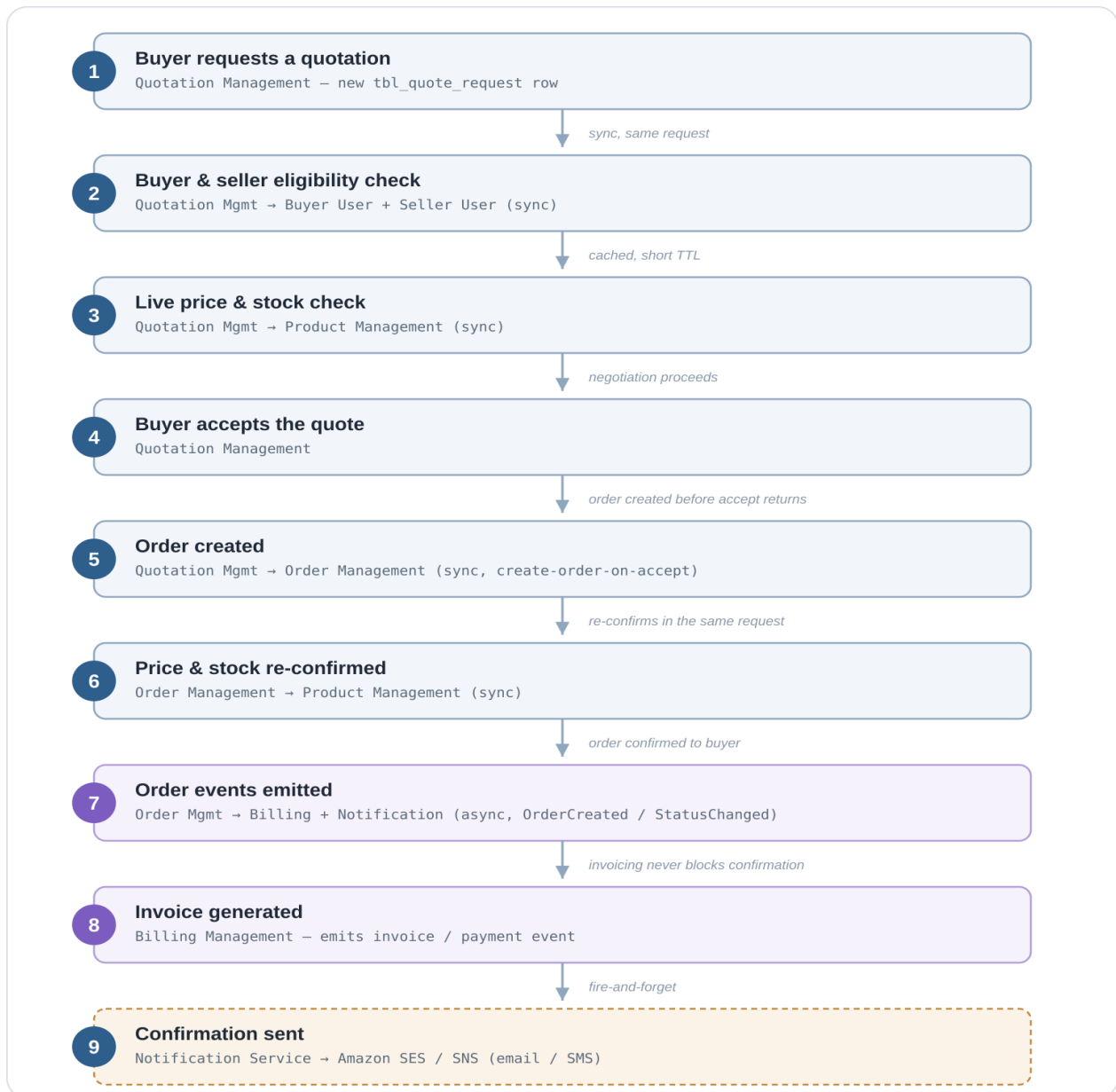


Figure 4 — Order creation (step 5) happens synchronously inside the quote-acceptance request per ADR-003 billing and notification (7–9) run asynchronously afterward and never block order confirmation.

10. Integration Architecture

See Figure 1 (Section 6) for the external integration boundary. The table below is the complete internal and external call matrix, including protocol and authentication for each.

System / API	Type	Purpose	Protocol / Method	Authentication
Client → API Gateway	External	All reads/writes across the platform	HTTPS / REST	Cognito / JWT authorizer (Assumption)
API Gateway → domain service	Internal	Routes matched requests to the target service	HTTPS / REST	Internal service role (IAM)
Order Mgmt → Seller User	Internal	Seller active/approved check	REST / HTTPS (sync)	Internal, Cloud Map resolved
Order Mgmt → Buyer User	Internal	Buyer account active check	REST / HTTPS (sync)	Internal, Cloud Map resolved
Order Mgmt → Product Mgmt	Internal	Confirm current price/stock	REST / HTTPS (sync)	Internal, Cloud Map resolved
Quotation Mgmt → Seller/Buyer User	Internal	Eligibility checks	REST / HTTPS (sync)	Internal, Cloud Map resolved
Quotation Mgmt → Product Mgmt	Internal	Pricing lookup	REST / HTTPS (sync)	Internal, Cloud Map resolved
Quotation Mgmt → Order Mgmt	Internal	Create order on quote acceptance	REST / HTTPS (sync)	Internal, Cloud Map resolved
Product Mgmt → Seller User	Internal	Seller-approval check before listing publishes	REST / HTTPS (sync)	Internal, Cloud Map resolved
Billing Mgmt → Order Mgmt	Internal	Line-item lookup at invoice render time	REST / HTTPS (sync, on demand)	Internal, Cloud Map resolved
Order Mgmt → Billing / Notification	Internal	OrderCreated / OrderStatusChanged	Event (EventBridge, async)	IAM event-bus policy
Billing Mgmt → Notification	Internal	Invoice generated / payment failed	Event (EventBridge, async)	IAM event-bus policy
Product Mgmt → Search Service	Internal	ProductChanged — index rebuild		IAM event-bus policy

System / API	Type	Purpose	Protocol / Method	Authentication
All services → Discovery Service	Internal / Platform	Register on boot, heartbeat, resolve peers	Event (EventBridge, async — Kafka candidate)	IAM execution role
All services → Config Service	Internal / Platform	Config pull at boot, refresh on event	HTTP pull + webhook refresh	IAM execution role
Notification Service → Amazon SES	Third-party (AWS)	Transactional email delivery	AWS SDK / API	IAM role
Notification Service → Amazon SNS	Third-party (AWS)	SMS delivery	AWS SDK / API	IAM role
Payment gateway → Billing Mgmt	Third-party	Not named in current v0.2 scope	To Be Confirmed	To Be Confirmed

11. Deployment Architecture

Fully cloud-based on AWS — no on-premises components, and no hybrid deployment is in scope for v0.2. Every service is provisioned by **one Terraform module per service**, with region passed in as a variable and remote state isolated per region (ADR-006) the existing GitHub Actions pipeline gains a region workflow_dispatch input rather than being duplicated per region.

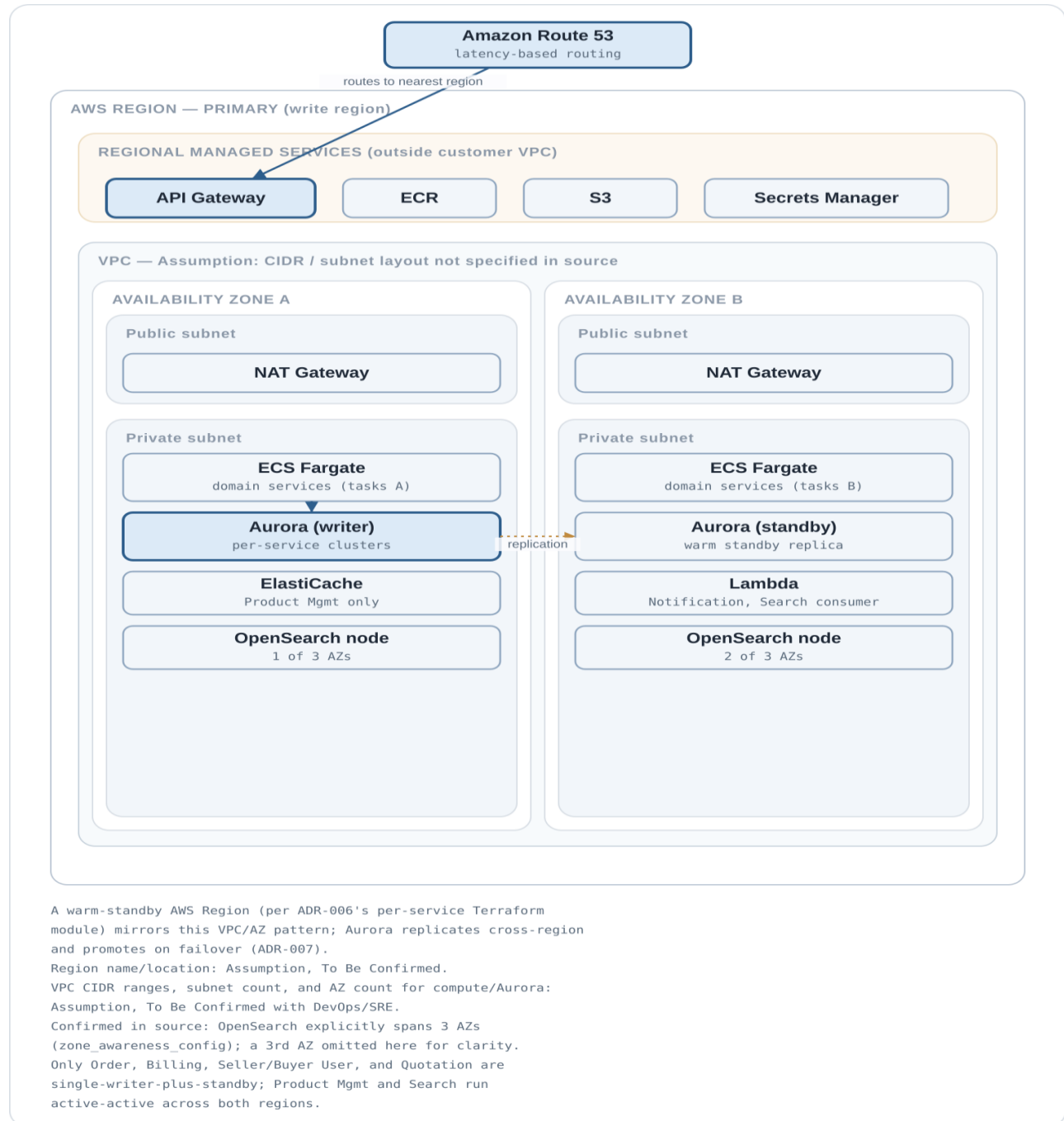


Figure 5 — AWS deployment and network topology. Compute splits by traffic shape: ECS Fargate for warm, steady-load services AWS Lambda for bursty, event-driven work. The VPC/subnet/AZ layout shown is the standard AWS pattern implied by the service Terraform's use of `var.vpc_id` and `var.private_subnet_ids` — exact CIDR ranges and AZ counts are Assumption / To Be Confirmed with DevOps/SRE. OpenSearch's 3-AZ zone-awareness is the one AZ count explicitly stated in source.

Region topology follows ADR-007 directly: single-writer-plus-standby for Order, Billing, Seller User, Buyer User, and Quotation active-active for Product Management and Search. A regional failure in a single-writer service means a controlled Aurora promotion (RTO seconds-to-minutes) — accepted in exchange for payment/order correctness.

Environments. Not itemized in the current source material beyond production and its warm-standby region a conventional dev / QA / staging / production progression is assumed but not formally specified (Section 4).

12. Security Architecture

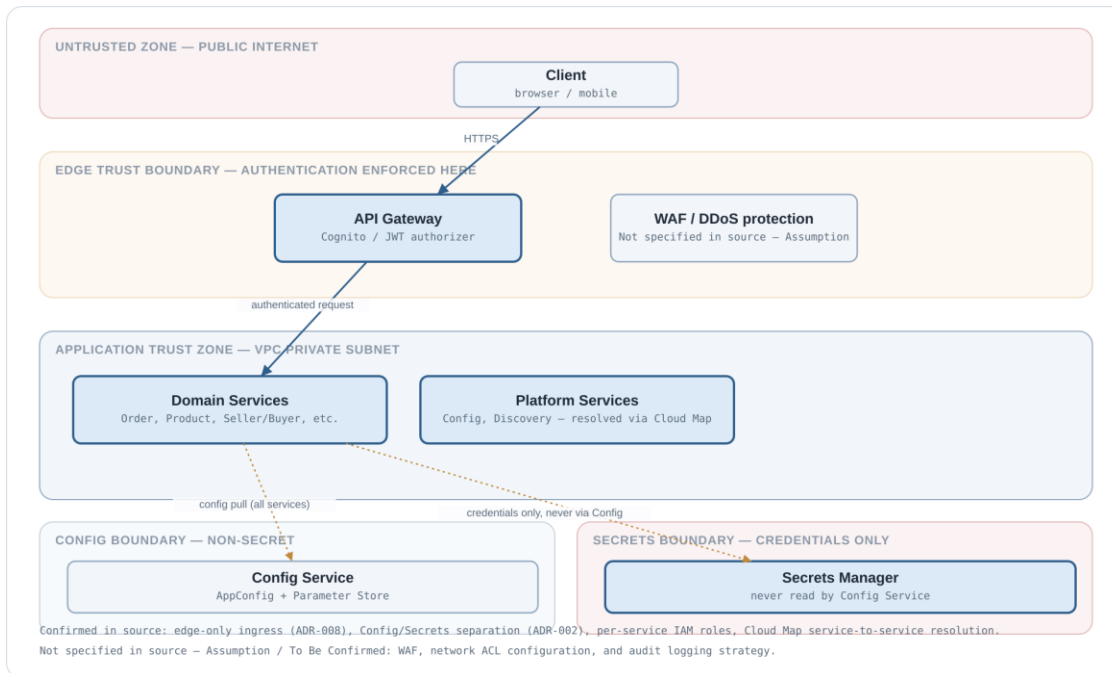


Figure 6 — Trust boundaries and security zones. The API Gateway boundary and the Config/Secrets separation are both explicit architecture decisions (ADR-008, ADR-002) the WAF box and network-ACL detail are marked Assumption since the source material does not specify them.

- **Authentication and authorization:** enforced once, at the edge — API Gateway is the sole client-facing entry point (ADR-008) with a Cognito/JWT authorizer (Assumption) no domain service is directly reachable from the internet.
- **Data encryption:** every Aurora cluster in the appendix Terraform sets `storage_encrypted = true` OpenSearch domains enable `encrypt_at_rest` and

node_to_node_encryption. In-transit encryption is assumed standard (HTTPS/TLS) for all AWS-managed endpoints but is not separately itemized in source.

- **Secrets management:** a hard, structural boundary — Config Service holds only non-secret values AWS Secrets Manager is the only place credentials are ever read from (ADR-002). A compromised Config Service read path can leak configuration, but never credentials.
- **Network security:** API Gateway as the single ingress point is the primary network control described in source VPC/private-subnet placement is conventional but unconfirmed in detail (Section 11). No WAF/DDoS configuration is named — **To Be Confirmed**.
- **Audit logging and monitoring:** **not specified** in the source material — see Section 14.
- **Compliance requirements:** **none named** in source data-privacy/drug-sale regulatory requirements are plausible given the domain but unconfirmed.

13. Scalability & Availability

CURRENT ARCHITECTURE — as documented in source:

- Autoscaling triggers are matched per service to what actually constrains it, not a blanket CPU rule: request-queue depth for I/O-bound document upload (Seller/Buyer User), read volume for Product Management, event/message backlog for Lambda consumers (Notification, Search indexer).
- **Statelessness:** ECS Fargate services are horizontally scalable behind Cloud Map resolution — no session affinity is described.
- **Database scaling:** Aurora Global Database gives Product Management active-active read replicas across regions all other transactional services are single-writer with a warm standby (ADR-007) — a deliberate scalability/consistency trade-off, not an oversight.
- **Caching:** Amazon ElastiCache (Redis) in front of Product Management's Aurora cluster absorbs high-volume catalog reads — no other service gets this treatment, because no other service has Product's read volume. Short-TTL caches sit on synchronous lookup calls elsewhere (Seller/Buyer approval-status checks, Billing's line-item lookup), explicitly never as a persisted copy.
- **Async/queue-based processing:** EventBridge + SQS decouples Billing and Notification from Order's write path entirely Lambda-based consumers scale to zero between bursts.
- **Load balancing:** handled by the managed API Gateway and Cloud Map at the service-discovery layer no separate ALB/NLB is described in source for internal traffic.

Single points of failure identified:

- API Gateway is a single dependency for all client traffic (mitigated by AWS's multi-AZ SLA and by keeping service-to-service traffic off this path — ADR-008).

- Each single-writer service (Order, Billing, Seller/Buyer User, Quotation) has exactly one writer at a time by design — failover is a controlled promotion (RTO seconds-to-minutes), not instant.
- Billing's invoice rendering has a runtime dependency on Order Management being reachable (mitigated by a short-TTL cache, per ADR-004).

RECOMMENDED ARCHITECTURE / OPEN ITEMS — not present in source, flagged for follow-up rather than assumed:

- Formal performance SLOs (response time, throughput, latency targets) are not defined — recommend establishing per-service SLOs before production launch.
- Expected user and transaction volumes are not quantified anywhere in the source material — sizing decisions (instance counts, autoscaling thresholds) cannot be fully validated without them.

14. Observability & Monitoring

To Be Confirmed — not specified in the source material. No logging aggregation, metrics/dashboarding, tracing, or alerting strategy is described for any of the 11 services. This is flagged as an open item in Section 18 (Risks) rather than silently assumed. Amazon CloudWatch is the natural default given the platform's AWS-native bias elsewhere (ADR-002), but this is a recommendation, not a documented decision.

15. Disaster Recovery / Backup

Backup retention is explicitly configured in the appendix Terraform: **7 days** for Seller User, Buyer User, Order Management, and Quotation Management **14 days** for Billing Management, reflecting its financial/ledger data. Recovery for single-writer services is a controlled Aurora promotion to the standby region, with an accepted RTO of seconds-to-minutes (ADR-007) — not instant failover, traded deliberately for write correctness.

To Be Confirmed: formal RPO targets, backup testing/restore drills, and a documented disaster-recovery runbook are not present in the source material.

16. CI/CD Architecture

One Terraform module per service, region passed in as a variable, remote state isolated per region (ADR-006). The existing GitHub Actions pipeline gains a region `workflow_dispatch` input rather than being duplicated per region — adding a region is one parameterized pipeline run per service, not a manual setup.

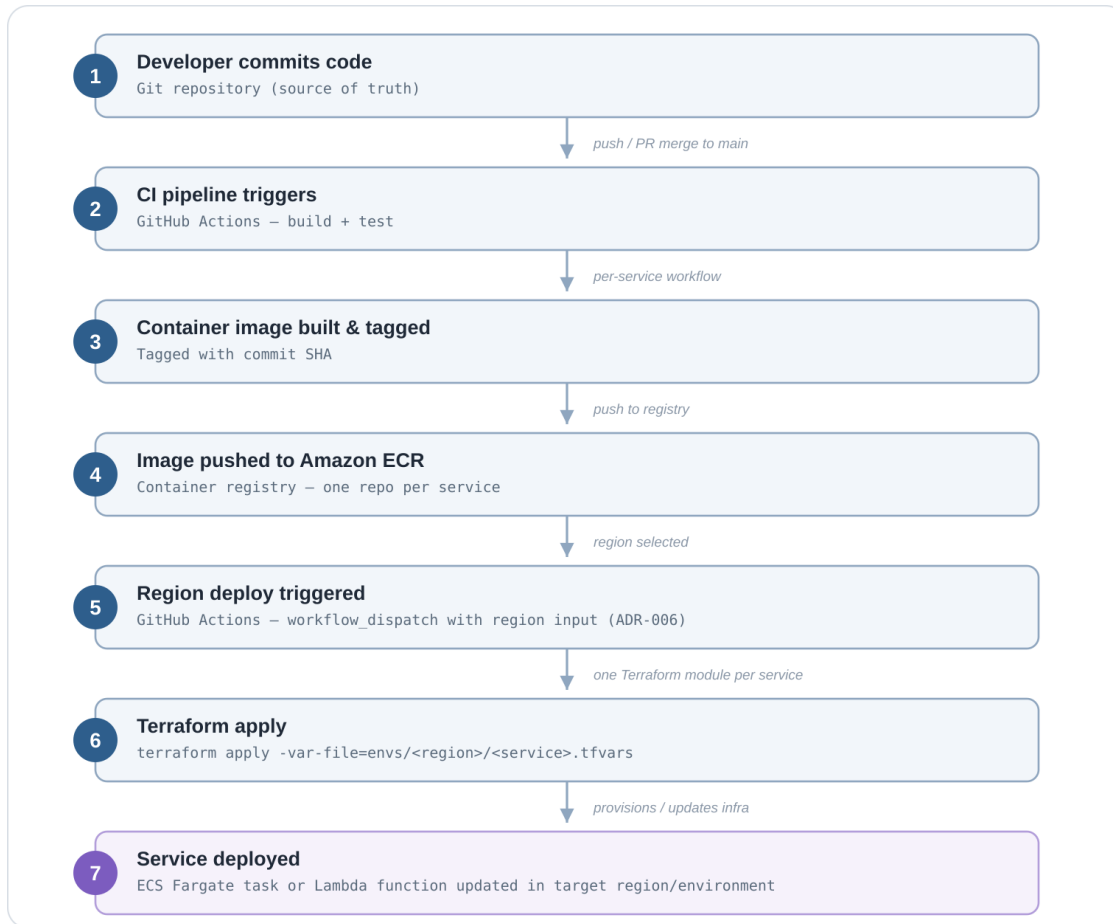


Figure 7 — CI/CD pipeline. Steps 1–4 (build/test/registry) are described directly in source step-level detail beyond "GitHub Actions" (e.g. specific test gates, approval steps) is not further specified.

17. Non-Functional Requirements

Category	Requirement / target	Status
Availability	Single-writer services: seconds-to-minutes RTO via controlled Aurora promotion. Active-active services (Product, Search): continuous multi-region availability.	Documented (ADR-007)
Performance	No formal response-time/throughput/latency SLOs specified	Confirmed
Scalability	Horizontal, per-service, trigger matched to actual constraint (queue depth, read volume, event backlog)	Documented
Security	Single ingress point, Config/Secrets separation, encryption at rest for Aurora/OpenSearch	Documented (partial — see Section 12 gaps)
Maintainability	One Terraform module per service consistent per-service ownership boundaries	Documented (ADR-006)
Observability	No logging/monitoring/tracing strategy specified	Confirmed
Compliance	No framework named	Confirmed

18. Risks & Constraints

Risk / Constraint	Probability	Impact	Mitigation	Owner
Product Management extraction touches nearly every other service's product_id foreign keys	High	High	Product's own table split is straightforward the FK migration is sequenced and tracked separately (Appendix C, Product Management)	Catalog & Search Team
Constraint: Aurora PostgreSQL supports only one writer region at a time	—	High	Directly forces single-writer-plus-standby topology for write-sensitive services (ADR-007), not a preference	Order & Commerce Team
tbl_terms_master lives under entity/temp/seller today but its	Medium	Medium	Move to a shared legal-content table referenced by both Seller User and Buyer User, not owned by either	Identity & Accounts Team

Risk / Constraint	Probability	Impact	Mitigation	Owner
content isn't seller-specific				
EventBridge does not guarantee per-key ordering or cheap replay	Medium	Medium	Flagged as a named Kafka (MSK) candidate (ADR-009) — migrate only that one edge, once real event volume is measured	Catalog & Search Team
Billing's invoice rendering has a runtime dependency on Order Management being reachable	Low	Medium	Short-TTL cache on the line-item lookup (never persisted, per ADR-004)	Billing & Notifications Team
API Gateway is a single dependency for all client traffic	Low	High	AWS's own multi-AZ SLA service-to-service calls kept off this path entirely (ADR-008)	Platform Engineering Team
A compromised Config Service read path	Low	Medium	Secrets structurally excluded from Config Service's store (Secrets	Platform Engineering Team
Constraint: platform layer (Gateway, Config, Discovery) must be operational before any domain split delivers value	—	Medium	Platform-first sequencing is mandatory in the migration roadmap (ADR-001)	Platform Engineering Team
No named external payment gateway	Medium	Medium	Called out explicitly as an open assumption rather than silently designed around a guess	Billing & Notifications Team
No audit logging / monitoring or compliance framework defined yet	Medium	High	Flagged for a dedicated security and compliance review before production launch	Security / Platform Engineering

19. Future Scalability

- The Product→Search edge is the one path pre-flagged to move from EventBridge to Amazon MSK if/when real event volume demands ordered replay (ADR-009) — evaluated on measured data, not preemptively.
- Region expansion beyond the first additional region is explicitly deferred from full orchestration tooling (AWS Control Tower/Landing Zone) until region count grows past a handful (ADR-006) the current parameterized-Terraform approach is scoped for near-term growth, not large-scale sprawl.
- As Quotation Management and other services extract fully from the monolith, their cross-domain foreign keys become opaque, API-resolved references — a pattern already adopted, ready to extend to any future service split.

20. Appendix

The appendices carry the full supporting detail behind this document. Appendix A holds the architecture decision records this design rests on. Appendix B is the complete AWS service-by-service reference. Appendix C carries the detailed per-service design — data ownership down to the table level, deployment Terraform, and service-specific decisions — for all 11 services, including each service's own architecture diagram.

Appendix A — Architecture Decision Records

Nine decisions that shaped the v0.2 service split — what was decided, what it beat, and the trade-off accepted in exchange. Each links to the service section(s) it governs, further down this same document.

ADR-001 — Service boundaries, v0.2

Adopt 11 services — Seller User, Buyer User, Order Management, Quotation Management, Search Service, Product Management, Billing Management, Notification Service, API Gateway, Config Service, Discovery Service. Master/reference data stays a shared module, not a numbered service.

Context: v0.1 proposed 5 coarse domain services. Stakeholders want finer granularity to scale Order, Quotation, Billing, and Search independently, plus explicit platform services instead of implied infrastructure.

Alternatives considered: Keep v0.1's 5-service split or split further per product category (rejected — no evidence of independent scaling need per category, and it would fragment a self-contained domain for no operational benefit).

Trade-offs: More services to deploy, monitor, and secure individually, in exchange for independent scaling and clearer team ownership per capability.

Consequences: Requires the platform layer (Gateway/Config/Discovery) to be operational before any domain service split delivers value — sequence platform-first in the migration roadmap.

Governs:

[Seller User](#), [Buyer User](#), [Order Management](#), [Quotation Management](#), [Search Service](#), [Product Management](#), [Billing Management](#), [Notification Service](#)

ADR-002 — Discovery & Config: AWS-native over self-hosted

Use AWS Cloud Map for discovery and AWS AppConfig + Systems Manager Parameter Store for config, both provisioned per region via the same Terraform module.

Context: One-click multi-region requires services to find each other and get region-correct config without manual wiring.

Alternatives considered: Netflix Eureka + Spring Cloud Config Server HashiCorp Consul. Both are natural fits for a Spring Boot team, but each is a stateful cluster to run, patch, and replicate per region.

Reason: The backend already deploys via ECS/ECR — Cloud Map integrates with ECS service discovery natively, with no extra cluster to run per region.

Trade-offs: Ties discovery/config to AWS acceptable since compute, data, and CI/CD are already AWS-committed.

Governs: [Config Service](#), [Discovery Service](#)

ADR-003 — Sync vs async: default to async

A call is synchronous only when the caller cannot return a correct response without the result in the same request (Order↔Product stock/price check, Quotation↔Order creation on accept). Everything else — Billing, Notification, Search indexing — is event-driven over EventBridge.

Context: Cross-service coupling risk grows with every synchronous call added.

Alternatives considered: Sync everywhere (simpler to reason about, but couples the availability of every service to every caller) async everywhere (adds latency/complexity where a real-time answer is genuinely required).

Trade-offs: Async paths need idempotent consumers and a dead-letter queue per consumer sync paths need circuit breakers so a slow Product Management doesn't cascade into failed orders.

Governs: [Order Management](#), [Quotation Management](#), [Billing Management](#), [Notification Service](#), [Product Management](#)

ADR-004 — Billing owns no Order data copies

Billing's schema stores order_id as a foreign reference only. It's created from an OrderConfirmed event carrying just order id, buyer id, and total any line-item detail needed for invoice rendering is fetched synchronously from Order Management at render time, never persisted in Billing.

Context: Explicit requirement: Billing must not duplicate Order Management's data.

Alternatives considered: Full data copy via CDC (rejected — reintroduces the duplication being avoided) a shared database/schema (rejected — violates service data ownership and makes the boundary unenforceable).

Trade-offs: Invoice rendering has a runtime dependency on Order Management being reachable mitigate with a short-TTL cache on the line-item lookup, not a persisted copy.

Governs: [Billing Management](#), [Order Management](#)

ADR-005 — Search Service owns an index, not source data

Search Service has no source-of-truth tables — it maintains an OpenSearch index rebuilt from Product Changed events published by Product Management.

Context: Search needs to scale independently of Product Management's write path.

Alternatives considered: Query Product Management's DB directly for search (rejected — couples search query load to the transactional catalog DB) Product Management embeds search itself (rejected — different scaling profile, per ADR-001).

Trade-offs: Search results are eventually consistent with catalog changes — acceptable for browse/search, not used for stock/price truth (Order still calls Product Management directly for that).

Governs: [Search Service](#), [Product Management](#)

ADR-006 — One-click multi-region via parameterized IaC

One Terraform module per service, region passed as a variable, remote state isolated per region the existing GitHub Actions pipeline gains a region workflow_dispatch input instead of being duplicated per region.

Context: Requirement: adding a region should be a single scripted action, not manual per-service setup.

Alternatives considered: Manually provisioned regions (rejected — doesn't meet the one-click requirement and is error-prone at 11 services) a full orchestration platform like AWS Control Tower/Landing Zone (deferred — justified once region count grows past a handful, not needed for the first additional region).

Trade-offs: Requires discipline that nothing is provisioned by hand outside Terraform — any manual change in one region silently diverges from the module and breaks repeatability.

Governs: [API Gateway](#), [Config Service](#), [Discovery Service](#) — all services, per module

ADR-007 — Single-writer regions for Order, Billing, Seller/Buyer User, Quotation active-active for Product, Search, Master Data

Transactional/write-sensitive services (Order, Billing, Seller/Buyer User, Quotation) run single-writer with a warm standby region. Read-heavy, lag-tolerant services (Product Management, Search's OpenSearch index, Master/Reference Data on DynamoDB Global Tables) run active-active across regions.

Context: Aurora PostgreSQL Global Database supports one writer region at a time true multi-writer needs a different data model entirely.

Alternatives considered: Force everything active-active via DynamoDB (rejected — Order/Billing need relational transactions and strong consistency a key-value multi-writer store doesn't give for free) force everything single-writer (rejected — throws away the real multi-region win for the read-heavy services).

Trade-offs: A regional outage in the primary write region means a controlled Aurora promotion (seconds-to-minutes RTO) for Order/Billing, not instant failover — accepted in exchange for payment/order correctness.

Governs: [Order Management](#), [Billing Management](#), [Seller User](#), [Buyer User](#), [Quotation Management](#), [Product Management](#), [Search Service](#)

ADR-008 — API Gateway is the only client-facing entry point

Amazon API Gateway fronts all client traffic no domain service is directly reachable from the internet. Service-to-service calls (Order→Product, Quotation→Order) go direct via Cloud Map resolution, bypassing the gateway.

Context: 11 independently deployable services need one consistent place to enforce auth, rate limiting, and routing rather than each reimplementing it.

Alternatives considered: Self-hosted Spring Cloud Gateway on Fargate — kept as a fallback option if routing needs (e.g. complex request transformation) outgrow what managed API Gateway supports.

Trade-offs: API Gateway becomes a single dependency for all client traffic — mitigated by AWS's own multi-AZ SLA, and by keeping service-to-service calls off this path entirely so an internal domain issue doesn't compound with a gateway issue.

Governs: [API Gateway](#), [Discovery Service](#)

ADR-009 — Event backbone: EventBridge by default, Kafka only where replay/ordering/volume demand it

Every async edge in the communication matrix (ADR-003) runs on Amazon EventBridge, with SQS behind each consumer for retry/DLQ handling, unless a specific edge needs one of the properties EventBridge doesn't provide — in which case that ONE edge moves to Amazon MSK (managed Kafka), not the whole system.

Context: EventBridge and SQS cover fan-out and task-queue semantics with no cluster to run, matching ADR-002's AWS-native bias. But EventBridge does not guarantee per-key ordering, its Archive/Replay path is a slow, ops-triggered recovery tool rather than a routine replay mechanism, and its per-event pricing/throughput has a practical ceiling.

Decision — the four triggers. An async edge graduates from EventBridge to Kafka (MSK) only when it needs at least one of:

1. **Ordered replay is a correctness requirement** — out-of-order delivery silently corrupts state (not just delays it).
2. **Routine replay-from-any-offset** is part of normal operation, not disaster recovery — e.g. rebuilding a derived store from scratch.
3. **Multiple independent consumer groups** need to read the same stream at their own pace with their own offsets, without N hand-wired EventBridge targets.
4. **Sustained volume exceeds EventBridge's cost/throughput curve** — measured, not assumed.

Where this applies today: the Product Management → Search Service edge (ProductChanged) is the only edge in the current service catalog that plausibly hits triggers 1 and 2 — see [Search Service §02](#). This edge is flagged as a **Kafka candidate**, evaluated once real event volume is measured — not switched preemptively.

Alternatives considered: RabbitMQ — rejected for now, nothing in the current catalog needs exchange/topic routing beyond SQS. Kafka everywhere — rejected as premature most edges are simple fan-out with no ordering/replay need.

Trade-offs: A mixed backbone means two operational models instead of one — accepted because forcing every edge onto the hardest edge's technology is the more expensive mistake. Revisit per-edge, not system-wide.

Governs: [Search Service](#), [Product Management](#), [Order Management](#), [Billing Management](#), [Notification Service](#)

Appendix B — AWS Services Reference

Every AWS service named across the 11-service architecture, why it was picked, and which service(s) it belongs to.

- **19** — Services in active use
- **1** — Flagged candidate (MSK)
- **8** — Categories
- **11** — Services documented

Categories: Compute, Database, Storage, App Integration, Networking, Security & Identity, Management & Governance, Developer Tools

Compute

Two compute types cover the whole system, split by traffic shape rather than convenience — see the Decisions section of [Search Service](#) for the reasoning this pattern comes from.

Amazon ECS on Fargate — compute · serverless containers · Active

Used by: Seller User, Buyer User, Order Mgmt, Quotation Mgmt, Billing Mgmt, Product Mgmt, Search (query layer)

The default for anything needing a **warm process** — a live connection pool to Aurora, low-latency request handling, or steady background load. No servers to patch, no idle-capacity waste at this traffic level.

AWS Lambda — compute · event-driven functions · Active

Used by: Notification Service, Search (index consumer)

Reserved for work that's **bursty and idle most of the time** — a catalog change, a notification send. Paying for an always-on Fargate task here isn't justified Lambda scales to zero between bursts.

Database & caching

Transactional services get a single-writer Aurora cluster each (per [ADR-007](#)) Product Management is the one exception, running active-active because it's read-heavy and lag-tolerant.

Amazon Aurora PostgreSQL — database · single-writer cluster, one per service · Active

Used by: Seller User, Buyer User, Order Mgmt, Quotation Mgmt, Billing Mgmt

Relational, transactional correctness where it actually matters — orders, payments, registrations. Single-writer with a warm standby, never active-active, per [ADR-007](#). **Billing**

runs a physically separate cluster from Order — enforcing the no-duplication rule ([ADR-004](#)) at the infrastructure level, not by convention.

Aurora Global Database — database · active-active reads across regions · Active

Used by: Product Management

The one write-heavy-but-read-tolerant domain. One write primary, but read replicas fan out globally — the exception to single-writer specifically because Product Management is 59% of the system's tables and almost entirely read traffic.

Amazon ElastiCache (Redis) — database · in-memory cache · Active

Used by: Product Management

Sits in front of Aurora specifically to absorb high-volume browse/catalog-read traffic without hitting the writer. No other service gets this treatment — no other service has Product's read volume.

Amazon OpenSearch Service — database · managed search index · Active

Used by: Search Service

Purpose-built full-text/faceted search. No other service touches it — it's a **disposable index** rebuilt from Product's events ([ADR-005](#)), never a source of truth. Managed snapshots and zone-aware replicas remove exactly the operational surface a search cluster otherwise demands.

App integration & messaging

Async-by-default ([ADR-003](#)) runs on EventBridge + SQS everywhere except one flagged edge — see [ADR-009](#) for exactly when that changes.

Amazon EventBridge — app integration · event bus · Active

Used by: Order → Billing, Order → Notification, Billing → Notification, Product → Search

The async backbone for everything per [ADR-003](#) — fan-out to independent consumers with no cluster to run, no polling.

Amazon SQS — app integration · queue, behind every consumer · Active

Used by: Every async consumer

Retry and dead-letter handling behind every EventBridge target — named explicitly in [ADR-009](#) so "async by default" doesn't silently drop failed deliveries.

Amazon SNS — app integration · SMS delivery · Active

Used by: Notification Service

SMS fan-out from the one service that never receives a synchronous call by design — Notification only subscribes to events, it exposes no internal REST endpoint at all.

Amazon SES — app integration · email delivery · Active

Used by: Notification Service

Email delivery — order confirmations, invoice notices — fired asynchronously from Billing/Order events, never on a critical path.

Amazon MSK (Kafka) — app integration · flagged, not adopted · Candidate

Used by: Product → Search (candidate)

The **only** edge in the system flagged for ordered, replayable delivery — Search's index-backfill replays this exact stream, and EventBridge doesn't guarantee per-key ordering. Named in [ADR-009](#).

Why not Kafka or RabbitMQ everywhere: most async edges here are simple fan-out with no ordering or replay requirement — running a managed Kafka cluster for those adds real operational cost with no matching benefit. RabbitMQ was considered and rejected outright: nothing in this system needs AMQP-style exchange/topic routing beyond what SQS already provides. See [ADR-009](#) for the full reasoning.

Platform & management

Config and Discovery sit underneath every service, not on the client request path — every choice here traces back to one rule: the backend already deploys via ECS/ECR, so pick whatever integrates with that natively ([ADR-002](#)).

AWS Cloud Map — management · service registry · Active

Used by: Discovery Service

Registration, health checks, DNS-based resolution for every service. Chosen over self-hosted Eureka/Consul specifically because it integrates natively with ECS — no new cluster to run per region, per [ADR-002](#).

AWS AppConfig — management · live config, no redeploy · Active

Used by: Config Service

Per-service, per-environment, per-region config with a refresh event instead of polling — this is the specific mechanism that makes the one-click region rollout in [ADR-006](#) work without touching running services.

Systems Manager Parameter Store — management · plain config values · Active

Used by: Config Service

Plain (non-secret) config values that AppConfig profiles reference — kept distinct from Secrets Manager below as a deliberate security boundary, not an accident.

Security & identity

AWS Secrets Manager — security · credentials, never in Config Service · Active

Used by: Config Service (boundary)

A deliberate boundary stated explicitly in the Config Service doc — secrets never live in Config Service's own store. Separating "config" from "credentials" is a security decision, not an oversight.

AWS IAM — security · roles & permissions · Active

Execution roles for Lambda, cross-service permission boundaries — referenced explicitly in Notification's Terraform, implicit in every other service's task role.

Networking & client-facing edge

Amazon API Gateway — networking · sole client entry point · Active

Used by: API Gateway service

Managed auth, rate-limiting, and routing out of the box. No domain service is directly reachable from the internet, per [ADR-008](#). Service-to-service calls bypass it entirely, resolving via Cloud Map instead.

Amazon Route 53 — networking · latency-based routing · Active

Used by: API Gateway (multi-region)

Routes clients to the nearest regional gateway. Keeps the region-add story a single parameterized deploy instead of a shared global resource to coordinate.

Amazon S3 — storage · onboarding documents · Active

Used by: Seller User, Buyer User

Onboarding document uploads — I/O-heavy, not CPU-heavy, which is also why these two services scale on request-queue depth rather than CPU utilization.

Developer tools

Amazon ECR — developer tools · container registry · Active

Every container image lands here before deploy — the existing build→ECR→ECS pipeline the backend already runs, referenced in every service's Terraform build-time section.

Appendix C — Detailed Service Design Documents

01 · Domain service

Seller User Service — schema consolidation and deployment topology

What Seller User owns today across three parallel entity packages, how those collapse into one service schema, and why this service sits in the single-writer bucket rather than active-active.

01. Overview

Seller User owns the seller's identity end to end: profile, registration, onboarding documents, bank details, and the seller/coordinator auth surface that Order, Quotation, and Product all lean on to answer "is this seller allowed to act." Nothing about a seller's catalog or transactions lives here — only who they are and whether they're approved.

Traffic is bursty rather than steady: registration and document upload spike around onboarding windows and go quiet in between. Document upload in particular is I/O heavy, not CPU heavy, so it scales on request queue depth, not CPU utilization.

02. Data ownership

The current monolith spreads seller data across three entity packages — roughly 21 tables — that this service must absorb into one schema.

Table	Package	Note
tbl_seller	entity/seller	Live, approved seller record.
tbl_seller_gst	entity/seller	GST registration details for a live seller.
tbl_seller_document	entity/seller	Approved onboarding documents, post-verification.
tbl_seller_bank_details	entity/seller	Live bank/payout details, validated against IFSC lookup.
tbl_seller_address	entity/seller	Registered business address(es).
tbl_seller_coordinator	entity/seller	Named coordinator/contact person, shares the auth surface below.
tbl_seller_history	entity/seller	Audit trail of changes to a live seller record.
tbl_pending_seller	entity/seller (profile)	Seller record awaiting admin approval.

Table	Package	Note
tbl_pending_seller_document	entity/seller (profile)	Documents attached to a pending-approval seller.
tbl_temp_seller	entity/temp/seller	In-progress registration, before submission.
tbl_temp_seller_document	entity/temp/seller	Documents uploaded mid-registration.
tbl_temp_seller_bank_details	entity/temp/seller	Bank details captured during registration.
tbl_temp_seller_coordinator	entity/temp/seller	Coordinator details captured during registration.
tbl_temp_seller_address	entity/temp/seller	Address captured during registration.
tbl_temp_seller_email_otp	entity/temp/seller	Email verification OTP.
tbl_temp_seller_review_history	entity/temp/seller	Reviewer actions/comments on a temp registration.
phone_otp	entity/temp/seller	Phone verification OTP.
tbl_terms_master	entity/temp/seller	Legal terms content — misplaced, not actually seller-specific.
tbl_user	entity/auth	Shared login identity, seller and coordinator accounts.
tbl_signup_otp	entity/auth	Signup OTP, shared auth table.
tbl_refresh_tokens	entity/auth	Refresh token store, shared auth table.
tbl_login_otp	entity/auth	Login OTP, shared auth table.

Open question — tbl_terms_master is misplaced: it lives under entity/temp/seller today but its content is shared, not seller-specific. It belongs in a shared legal-content table, referenced by both Seller User and Buyer User rather than owned by either.

Three lifecycles, one schema: a seller record moves through **Temp → Pending → live Seller** across three separate table families with independent primary keys. Consolidating means picking a single seller identity that persists across all three stages (a state column, not three tables).

Also flag: tbl_ifsc_overrides sits in its own entity/ifsc package today but exists purely to validate bank details — a candidate to fold into Seller User's schema.

Both Seller User and Buyer User FK into shared master tables — tbl_state_master, tbl_district_master, tbl_taluka_master,

tbl_seller_type_master/tbl_buyer_type_master, tbl_document_type_master. These stay a shared read-only reference module, cached at the edge.

03. Communication

Seller User is called far more than it calls out — the dominant traffic pattern is other services asking "is this seller approved."

Sync Inbound eligibility lookups

on nearly every write path elsewhere

- Order Management → Seller User: is this seller active/approved.
- Quotation Management → Seller User: same eligibility check.
- Product Management → Seller User: seller-approval check before a listing can be published.
- Read-only, high frequency — cache aggressively, short TTL, invalidated on approval-status change.

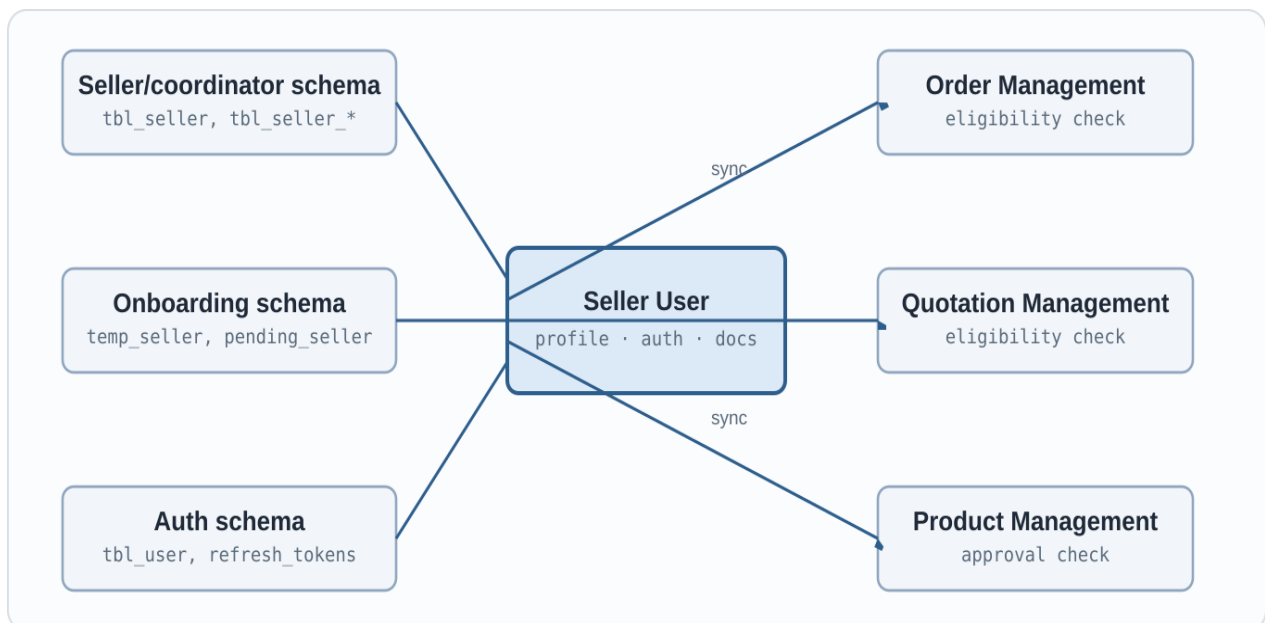
Platform Boot-time platform calls

on service startup, not business traffic

- Registers with Discovery Service so callers can resolve it.
- Pulls environment config from Config Service at boot.

04. Architecture diagram

Three callers doing high-frequency eligibility checks against one service, backed by its consolidated data.



Legend: sync request (eligibility/approval check) async / platform (not shown — Discovery + Config at boot)

05. Deployment

ECS Fargate for the API, Aurora PostgreSQL for the consolidated schema, S3 for document storage — the existing S3Service in the codebase already targets this pattern.

modules/seller-user-service/main.tf

```
// region as a variable, one module call per region – same pattern as Search
resource "aws_rds_cluster" "seller_user" {
  cluster_identifier = "seller-user-${var.region_short}-${var.environment}"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  master_username    = var.db_master_username
  master_password    = var.db_master_password
  db_subnet_group_name = var.db_subnet_group
  vpc_security_group_ids = [aws_security_group.seller_user_db_sg.id]
  backup_retention_period = 7
  storage_encrypted     = true
}

resource "aws_rds_cluster_instance" "writer" {
  cluster_identifier = aws_rds_cluster.seller_user.id
  instance_class     = var.db_instance_class
  engine             = aws_rds_cluster.seller_user.engine
}

resource "aws_s3_bucket" "seller_documents" {
  bucket = "pharma-seller-docs-${var.region_short}-${var.environment}"
}

module "seller_user_ecs_service" {
  source      = "../ecs-fargate-service"
  service_name = "seller-user"
  image       = "${var.ecr_repo_url}:${var.image_tag}"
  desired_count = var.task_count
  cpu         = 512
  memory      = 1024
  autoscale_on = "request_queue_depth"
}
```

06. Scaling & region topology

Seller User sits in the **single-writer with warm standby** bucket, not active-active — the opposite topology from Product/Search.

Registration and auth are correctness-sensitive: two regions independently accepting registrations creates split-brain identity data that's expensive to reconcile. Failover is a controlled Aurora promotion, minutes not seconds.

- **Writer region floor:** 2 tasks — *Fargate HA minimum, autoscaling on request queue depth above that*
- **Standby region floor:** 1 task + replica — *warm enough to promote quickly*
- **Failover time:** minutes — *controlled Aurora promotion, not instant*

07. Decisions worth writing down

Decision — one seller identity, three lifecycle states — Collapse Temp/Pending/Live into a single schema with a status column

Model as one seller table carrying a lifecycle_status enum, with history preserved in the existing history-table pattern.

Decision — standardize on Buyer's per-domain auth pattern — Seller User should adopt dedicated auth tables, not keep sharing entity/auth

Seller forks its own auth tables scoped to seller + coordinator logins, following Buyer's already-proven pattern — a shared entity/auth package becomes a cross-service dependency once both are separate services.

Decision — fold IFSC overrides in, carve terms master out — tbl_ifsc_overrides moves into Seller User tbl_terms_master moves to shared legal content

tbl_ifsc_overrides has no other reader — belongs inside Seller User's schema.
tbl_terms_master should move to the shared reference module alongside state/district/document-type masters.

02 · Domain service

Buyer User Service — schema, deployment, and auth-pattern alignment

What Buyer User owns today, how it mirrors Seller User's shape at lower volume, and the auth-table inconsistency between the two domains.

01. Overview

Buyer User owns the buyer's identity: profile, registration, onboarding documents, and buyer auth. It's the same shape as Seller User, just lighter in volume — no GST/bank-details/coordinator surface.

Registration traffic is bursty around onboarding pushes document upload is I/O bound, so it scales on request queue depth. The difference from Seller User is scale, not architecture.

02. Data ownership

15 tables across two packages — simpler than Seller User's three-lifecycle split, because Buyer only has two stages.

Table	Package	Note
tbl_buyer	entity/buyer	Live, approved buyer account record.
tbl_buyer_user	entity/buyer	Login identity for a buyer.
tbl_buyer_address	entity/buyer	Registered address(es).
tbl_buyer_delivery_address	entity/buyer	Delivery-specific address(es).
tbl_buyer_contact	entity/buyer	Named contact person on the buyer account.
tbl_buyer_document	entity/buyer	Approved onboarding documents.
tbl_buyer_signup_otp	entity/buyer	Signup OTP, buyer-domain-specific.
tbl_buyer_login_otp	entity/buyer	Login OTP, buyer-domain-specific.
tbl_buyer_refresh_tokens	entity/buyer	Refresh token store, buyer-domain-specific.
tbl_temp_buyer	entity/temp/buyer	In-progress buyer registration.
tbl_temp_buyer_addresses	entity/temp/buyer	Address captured during registration.
tbl_temp_buyer_contact	entity/temp/buyer	Contact details captured during registration.
tbl_temp_buyer_document	entity/temp/buyer	Documents uploaded mid-registration.

Table	Package	Note
tbl_temp_buyer_review_history	entity/temp/buyer	Reviewer actions/comments on a temp registration.

Inconsistency worth naming: Buyer already has its own dedicated OTP/refresh-token tables, fully scoped to this domain. Seller reuses a generic shared entity/auth package instead. The recommendation is for Seller to adopt Buyer's pattern.

Buyer User FKs into the same shared master tables as Seller User, remaining a shared read-only reference module.

03. Communication

Same shape as Seller User: mostly called, rarely calls out.

Sync Inbound eligibility/profile lookups

on order and quotation write paths

- Order Management → Buyer User: is this buyer account active.
- Quotation Management → Buyer User: buyer-profile/eligibility check.
- Lower volume than Seller User's equivalent, same caching discipline.

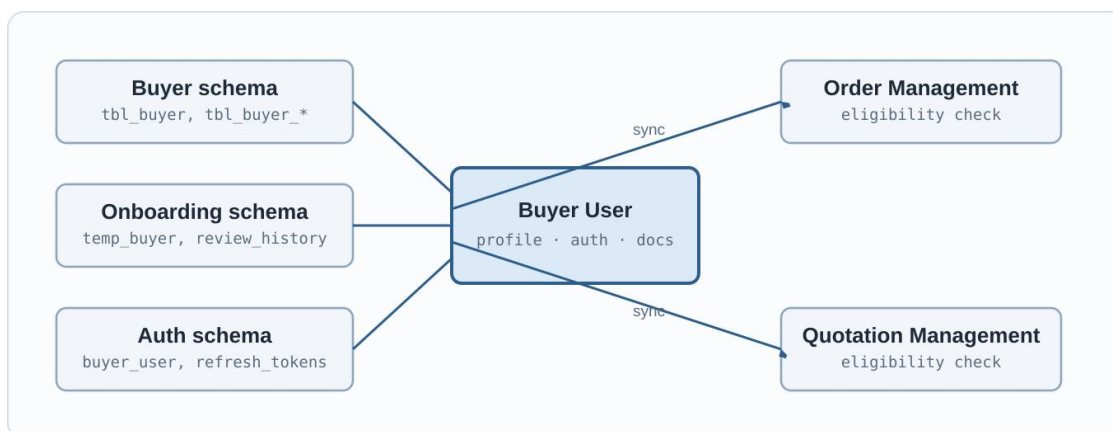
Platform Boot-time platform calls

on service startup, not business traffic

- Registers with Discovery Service.
- Pulls config from Config Service at boot.

04. Architecture diagram

Two inbound callers instead of Seller User's three — no Product-side gating check on the buyer side.



Legend: sync request (eligibility/profile check) async / platform (not shown — Discovery + Config at boot)

05. Deployment

Same module shape as Seller User, sized down — lower `desired_count` and a smaller DB instance class.

modules/buyer-user-service/main.tf

```
// near-identical to modules/seller-user-service/main.tf — lower task_count and instance_class
resource "aws_rds_cluster" "buyer_user" {
  cluster_identifier      = "buyer-user-${var.region_short}-${var.environment}"
  engine                  = "aurora-postgresql"
  engine_version          = "15.4"
  backup_retention_period = 7
  storage_encrypted       = true
}

module "buyer_user_ecs_service" {
  source      = "../ecs-fargate-service"
  service_name = "buyer-user"
  image       = "${var.ecr_repo_url}:${var.image_tag}"
  desired_count = var.task_count
  cpu          = 256
  memory       = 512
  autoscale_on = "request_queue_depth"
}
```

06. Scaling & region topology

Same **single-writer with warm standby** bucket as Seller User, for the same reason: correctness over read scale. Volume affects sizing, not the write-consistency requirement.

- **Writer region floor:** 1 task — *lower than Seller User's 2-task floor*
- **Standby region floor:** 1 task + replica — *kept warm for promotion*

07. Decisions worth writing down

Decision — collapse Temp/Live into one schema, same as Seller — Buyer's two-stage lifecycle gets one identity with a status column

One entity with a `lifecycle_status` column rather than two tables with independent keys.

Decision — Buyer's per-domain auth pattern is the one to standardize on — Keep Buyer's dedicated OTP/refresh-token tables Seller adopts this pattern, not the reverse

Buyer User Service ships its auth tables as-is it isn't the side with migration work to do.

Decision — no IFSC or terms-master ownership on the Buyer side — tbl_ifsc_overrides stays with Seller tbl_terms_master goes to shared legal content

Both Buyer User and Seller User should read terms from a shared legal-content table, not own a copy each.

03 · Domain service

Order Management — the strict-consistency core of the order lifecycle

Why Order stays single-writer while everything around it scales out, what has to physically move out of its package before Billing can be its own service, and how quote acceptance, price checks, and event fan-out fit together.

01. Overview

Order Management owns orders, order lines, order status transitions, and returns/refunds. This is the one service where correctness has to win over horizontal write scaling.

Double-charging a buyer, losing a status transition, or accepting an order against stock that no longer exists is not a data-quality problem, it's a financial and regulatory one. Order Management is deliberately single-writer and strongly consistent even though that costs active-active write scaling.

02. Data ownership

The entity/order package shows 8 tables: `tbl_order`, `tbl_seller_order`, `tbl_order_item`, `tbl_order_status_history`, `tbl_return_request`, `tbl_refund`, `tbl_payment`, and `tbl_invoice`.

The first six are squarely Order's: an order splits per seller since a single buyer cart can span multiple sellers, every status change is journaled rather than overwritten, and returns/refunds get their own tables.

Flag — `tbl_payment` and `tbl_invoice` are misplaced today: both currently live inside Order's package. This is a **required physical migration** into Billing's own schema and cluster before Billing can be extracted. See [Billing Management §02](#) and [ADR-004](#).

03. Communication

Order sits at the center: one sync call in from Quotation, one sync call out to Product, two async event streams to Billing and Notification.

Sync IN Quotation Management → Order

create-order-on-accept

- When a buyer accepts a quote, Quotation calls Order to create the order before returning "accepted."
- The order references the quote by ID nothing is copied into Order's tables.

Sync OUT Order → Product Management

price/stock confirmation on order creation

- Confirms current price and stock with Product Management in the same request.

Async OUT Order → Billing Management

OrderCreated / OrderStatusChanged via EventBridge

- Billing reacts to these events to generate invoices — invoicing never blocks order confirmation.

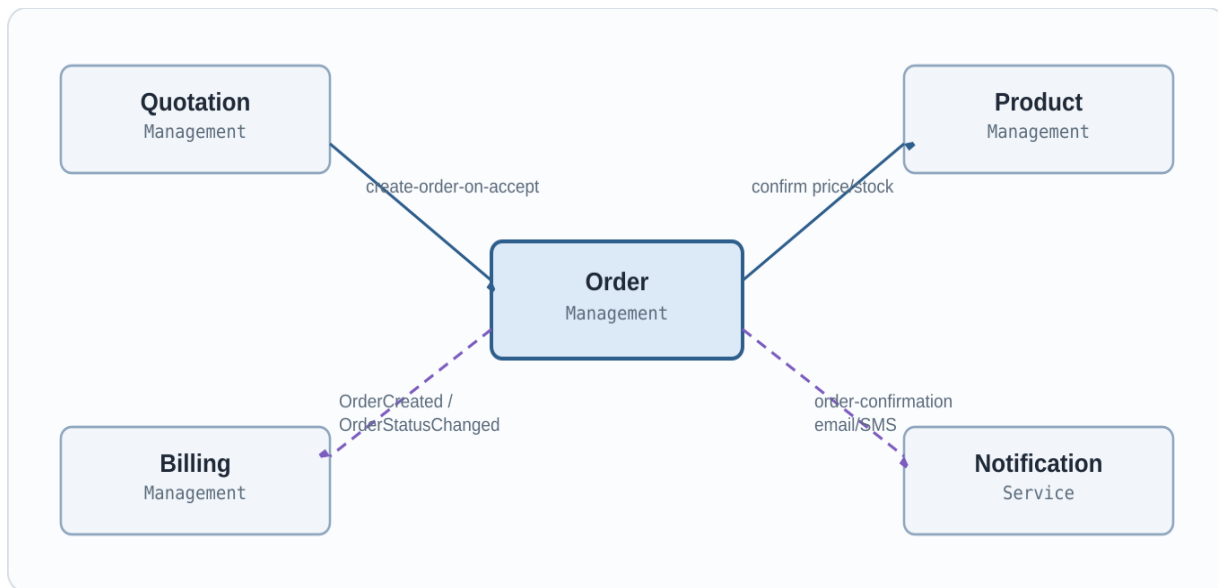
Async OUT Order → Notification Service

order-confirmation emails/SMS

- Notification must never receive a synchronous call — platform-wide rule.

04. Architecture diagram

Order Management as the hub: sync edges to Quotation and Product, async edges out to Billing and Notification.



Legend: sync (blocking) async (event)

05. Deployment

ECS Fargate for compute, Aurora PostgreSQL as a single-writer cluster — no multi-writer pattern.

modules/order-management/main.tf

```
// single-writer Aurora – standby region provisioned but not a second writer
resource "aws_rds_cluster" "order_db" {
  cluster_identifier      = "order-mgmt-${var.environment}"
  engine                  = "aurora-postgresql"
  engine_version          = "15.4"
  backup_retention_period = 7
}
```

```

module "order_ecs_service" {
  source          = "../ecs-fargate-service"
  service_name    = "order-management"
  image           = "${var.ecr_repo_url}:${var.image_tag}"
  desired_count   = var.task_count
  cpu             = 1024
  memory          = 2048
  autoscale_on    = "request_count"
}

variable "standby_region" {
  description = "region for warm-standby Aurora cluster promoted manually on regional failover"
  type        = string
}

```

06. Scaling & region topology

Order runs single-writer with a warm standby per [ADR-007](#) — explicitly **not** active-active.

A regional outage means a controlled Aurora promotion, RTO seconds-to-minutes. Active-active would mean resolving write conflicts on order and payment state, with no safe conflict-resolution strategy.

07. Decisions worth writing down

Decision — Payment and Invoice must be physically extracted — tbl_payment / tbl_invoice move to Billing's own schema/cluster

Extraction has to be physical — a shared table or view would leave the door open for a cross-service join, exactly what [ADR-004](#) prevents.

Decision — Order stays single-writer while Product goes active-active — Order correctness outranks read/write scale here

Per [ADR-007](#), Order is bucketed with the single-writer-plus-standby services because two regions independently accepting order writes creates exactly the failures this service is built to avoid.

04 · Domain service

Quotation Management — negotiation state and the single-table problem

Why one table backing two flows and three cross-domain foreign keys is a bigger problem than it looks, and how the negotiation trail should actually be modeled once this becomes its own service.

01. Overview

Quotation Management owns buyer quote requests, seller quote responses, and the negotiation state that sits between them — the pre-sale conversation that either dies quietly or converts into an Order.

Traffic tracks B2B negotiation cycles: quiet stretches interrupted by clusters of activity when a buyer organization is actively sourcing.

02. Data ownership

Exactly one table in entity/quote today: `tbl_quote_request`. No separate seller-response table, no separate negotiation-state table.

That one table backs **both** quoting flows — "Request Price Option" and RFQ "Get a Quote" — discriminated by a `requestType` enum.

Open architecture question, answered as a decision: a single mutable row overwritten on every back-and-forth loses the negotiation trail. Recommendation: a companion `tbl_quote_negotiation_history` table that appends one row per exchange.

Second flag — three-way cross-domain foreign keys: today's `QuoteRequest` entity directly foreign-keys `BuyerUser`, `Seller`, and `ProductDetails`. That breaks the moment those become separate service databases — the target service has to store opaque IDs and resolve via API calls.

03. Communication

Quotation makes two outbound synchronous calls and receives none.

Sync OUT Quotation → Product Management

pricing lookup during quote generation

- The buyer needs to see an actual number, not a stale indexed one.

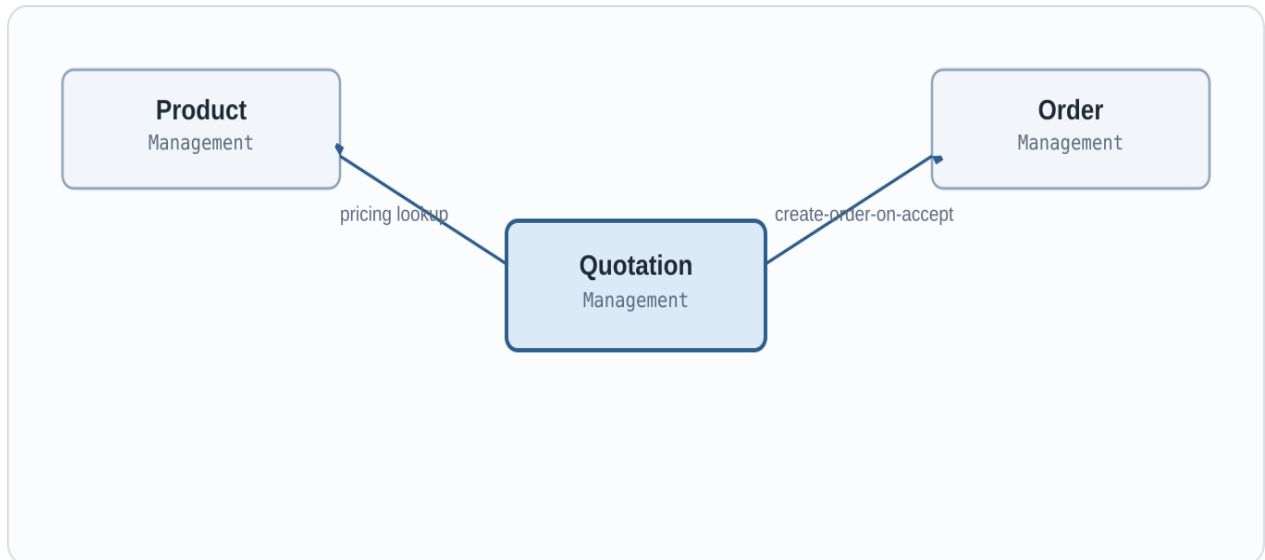
Sync OUT Quotation → Order Management

create-order-on-accept

- The order must exist before the acceptance response goes back to the buyer.

04. Architecture diagram

Quotation as a self-node with two synchronous outbound edges. No async edges.



Legend: sync (blocking)

05. Deployment

ECS Fargate, Aurora PostgreSQL — same engine as Order since a quote acceptance hands off synchronously.

modules/quotation-management/main.tf

```
// same single-writer-plus-standby shape as Order – see ADR-007
resource "aws_rds_cluster" "quote_db" {
  cluster_identifier      = "quotation-mgmt-${var.environment}"
  engine                  = "aurora-postgresql"
  engine_version          = "15.4"
  backup_retention_period = 7
}

module "quotation_ecs_service" {
  source      = "../ecs-fargate-service"
  service_name = "quotation-management"
  image       = "${var.ecr_repo_url}:${var.image_tag}"
  cpu         = 512
  memory      = 1024
  autoscale_on = "request_count"
}
```

06. Scaling & region topology

Single-writer with a warm standby, same bucket as Order Management, per [ADR-007](#) — same conflict-resolution reasoning as order state.

07. Decisions worth writing down

Decision — add a negotiation-history companion table — `tbl_quote_negotiation_history` instead of overloading one mutable row

Worth doing before Quotation splits into its own service — retrofitting history onto live negotiation data is much harder than starting with it.

Decision — opaque IDs instead of foreign keys across service boundaries — Buyer, Seller, and Product references become resolved-via-API, not FKs

A direct, mechanical consequence of the service-boundary split.

05 · Domain service

Search Service — build-time vs. deployment-time infrastructure

Where OpenSearch gets built, what Terraform provisions once versus per-deploy, and the AWS wiring that keeps the index disposable and the query path cheap.

01. Why this needs a build/deploy split at all

Search Service owns no source data — it's a query layer over an OpenSearch index rebuilt from ProductChanged events. The **index schema and mapping** is a build-time artifact the **index's data** is a runtime artifact rebuilt continuously and disposable.

02. Build-time: what CI produces

Build Query-layer image

on every merge to main

- Compile the stateless query service into one container image.
- Tag with commit SHA, push to ECR.

Build Index template & mapping

versioned alongside the consumer code

- A mapping change ships as a new index version (products-v13), never a mutation of the live index.
- CI validates the mapping against a sample document set before merge.

Why version the index name, not the domain: reindexing safely means writing to products-v13 while products-v12 still serves traffic, then flipping an alias.

EventBridge vs. Kafka for this edge, specifically: backfill works by replaying ProductChanged in order — EventBridge doesn't guarantee that. This is the one edge in the system flagged as a Kafka (MSK) candidate — see [ADR-009](#).

03. Deployment-time: what gets provisioned and reconciled

Trigger: git tag / workflow_dispatch (region as input param) → **Terraform apply:** OpenSearch domain (idempotent, per region) → **ECS deploy:** query-layer + consumer (pulls image from ECR) → **Bootstrap:** create products-vN, set alias (one-shot job, not Terraform) → **Backfill:** replay ProductChanged (from EventBridge archive or DB scan)

Deliberate boundary: Terraform provisions the OpenSearch *domain* — it does not create indices, mappings, or aliases. Index lifecycle is an application concern run by a bootstrap job.

04. Terraform module layout

modules/search-service/main.tf

```
// domain: infra only – no index/mapping resources live here
resource "aws_opensearch_domain" "search" {
  domain_name      = "search-${var.region_short}-${var.environment}"
  engine_version   = "OpenSearch_2.15"
  cluster_config {
    instance_type      = var.node_instance_type
    instance_count     = var.node_count
    zone_awareness_enabled = true
    zone_awareness_config { availability_zone_count = 3 }
  }
  ebs_options { ebs_enabled = true volume_type = "gp3" volume_size = var.volume_size_gb }
  vpc_options { subnet_ids = var.private_subnet_ids security_group_ids = [aws_security_group.search_sg.id] }
  encrypt_at_rest { enabled = true }
  node_to_node_encryption { enabled = true }
}

module "search_ecs_service" {
  source          = "../ecs-fargate-service"
  service_name    = "search-query-layer"
  image           = "${var.ecr_repo_url}:${var.image_tag}"
  desired_count   = var.query_layer_task_count
  cpu             = 512
  memory          = 1024
  autoscale_on    = "request_count"
}
```

Region rollout is one command: `terraform apply -var-file=envs/ap-southeast-1/search.tfvars`.

05. AWS service choices, and why

Layer	Choice	Alternative considered	Why this one
Index store	managed Amazon OpenSearch Service	Self-hosted OpenSearch on EC2/ECS	Managed snapshots, patching, zone-aware replicas remove exactly the operational surface a search cluster demands.
Query compute	managed ECS Fargate, stateless	Lambda per query	Query latency needs a warm connection pool Lambda cold starts would show up directly in result latency.

Layer	Choice	Alternative considered	Why this one
Index-build consumer	managed AWS Lambda, EventBridge-triggered	A standing Fargate consumer	Indexing is bursty and idle between catalog changes — pay-per-invocation beats a standing task.
Reindex/backfill job	self-run one-shot ECS task, run by CI on deploy	Terraform provisioner (local-exec)	Reindexing is an application action with retries and progress tracking.
Alias management	self-run bootstrap Lambda invoked post-deploy	Manual API call	Scripting the alias flip makes the one-click region story hold for Search too.

06. What a new region actually costs to spin up

- **Domain floor:** 3 nodes — *zone-aware across 3 AZs*
- **Cold-start cost:** ~0 — *index is empty until backfill runs*
- **Query layer:** 2 tasks — *scales on request volume post-launch*

Because the index rebuilds from events rather than replicating data, a new region needs no cross-region data transfer step at all.

07. Decisions worth writing down

Decision — index lifecycle is not a Terraform resource — Terraform owns the domain, application code owns the index

Keeps a mapping change on the same release cadence as the code that produces documents.

Decision — versioned index names, aliased in production — products-vN + alias, never a mutated live index

The query layer only ever queries the alias, never a version number directly.

Decision — consumer is Lambda, query layer is Fargate — Split compute by traffic shape, not by convenience

The indexing consumer tolerates cold start the query layer is latency-sensitive and needs a warm pool.

06 · Domain service

Product Management — the catalog of record and the cost of splitting it out

What Product Management owns today, how it's read by nearly every write path elsewhere, and why extracting it is the single biggest cross-cutting migration risk in the whole decomposition.

01. Overview

Product Management is the catalog of record for every sellable item — Drug, Consumable, Non-consumable, Supplement, Cosmetic, and Food/Infant, plus shared lookup masters.

Reads dominate by a huge margin, writes are comparatively rare. It's also the single largest domain by table count in the entire system.

02. Data ownership

A full entity audit counted **~62 entities/tables under entity/product/**** — roughly **59% of the system's ~106 tables**.

Group	Count	Representative tables
Core catalog	16	tm_product_details, tm_product_image, tm_pricing_details, tbl_stock_ledger, tbl_molecules_master, tm_category, tm_gst_percentage_master, and more
Per-category attribute tables	6	tm_product_attribute_drug, _consumable_medical, _non_consumable_medical, _supplements_or_nutraceuticals, _cosmetic_and_personal_use, _food_infant
Shared product-level lookup masters	12	ProductCategoryMaster, TherapeuticCategoryMaster, PackType, StrengthUnit, and more
MedicalDeviceProductMaster subpackage	14	DeviceCategory, MedicalDeviceType, StorageConditionMaster, Certification, and more
CosmeticPersonalCareMasters subpackage	4	SkinType, HairType, IntendedUseArea, ProductsFormMaster

The self-contained part is easy the referenced-from-outside part isn't: all 62 tables FK to each other and nothing outside entity/product/**. But **every other domain holds foreign keys into Product** — Quotation's and Order's line items both carry a product_id

that today resolves with a plain SQL join. That join disappears once Product moves to its own database.

03. Communication

Called synchronously by the two services with the sharpest latency budgets, and emits one async event stream outward.

Sync IN Order & Quotation Management

buyer is waiting in the same request

- Order calls in for price/stock confirmation before an order is placed.
- Quotation calls in for pricing when generating a quote.
- Mitigation lives on the caller's side: cache aggressively.

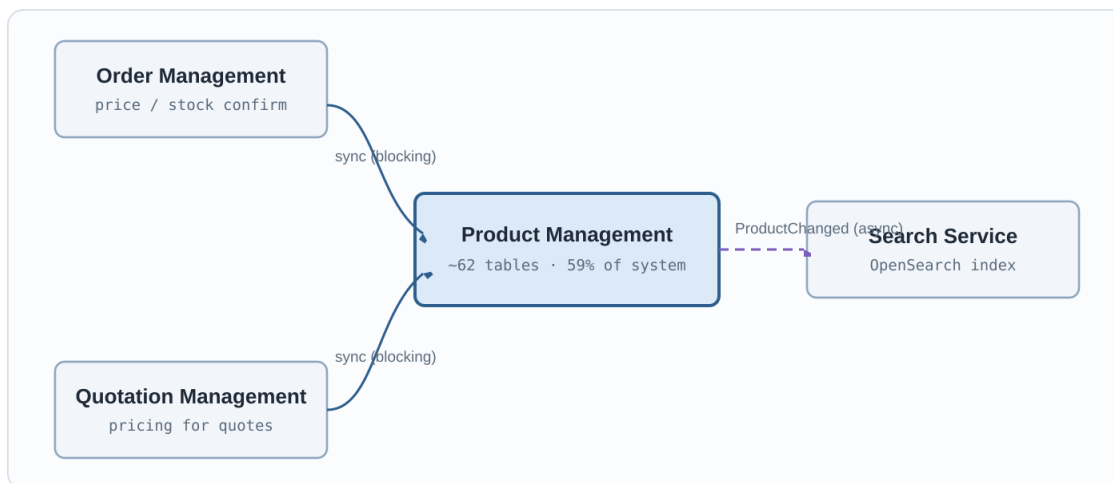
Async OUT Search Service

on every catalog change

- Emits ProductChanged events Search rebuilds its index, eventually consistent by design ([ADR-005](#)).
- Strictly one-directional — Search never calls back into Product.
- Flagged as the one Kafka (MSK) candidate — [ADR-009](#).

04. Architecture diagram

Product Management sits between two hard synchronous dependents and one async downstream consumer.



05. Deployment

ECS Fargate autoscaled on read volume — not CPU. Aurora PostgreSQL with a Global Database for multi-region reads, fronted by ElastiCache Redis.

```

modules/product-management/main.tf

// one global cluster, one primary write region
resource "aws_rds_global_cluster" "product" {
  global_cluster_identifier = "product-global"
  engine                    = "aurora-postgresql"
  engine_version            = "15.4"
}

resource "aws_rds_cluster" "product_regional" {
  cluster_identifier      = "product-${var.region_short}"
  global_cluster_identifier = aws_rds_global_cluster.product.id
  is_primary_cluster      = var.is_write_region
}

resource "aws_elasticache_replication_group" "product_cache" {
  replication_group_id      = "product-cache-${var.region_short}"
  engine                    = "redis"
  automatic_failover_enabled = true
}

module "product_ecs_service" {
  source      = "../ecs-fargate-service"
  service_name = "product-management"
  cpu         = 1024
  memory      = 2048
  autoscale_on = "request_count"
}

```

06. Scaling & region topology

Runs **active-active for reads** via Aurora Global Database, per [ADR-007](#) — a stale catalog read is a non-event.

- **Write region:** 1 primary — *single Aurora writer cluster*
- **Read regions:** N secondary — *Aurora Global Database secondary + ElastiCache per region*
- **Cache layer:** per-region — *sized to browse volume*

07. Decisions worth writing down

Decision — Product's own split is easy the cross-service FK migration is not — The 62 tables split cleanly every product_id foreign key elsewhere does not

The biggest cross-cutting migration risk anywhere in the decomposition — touches every consuming service, not just Product.

Decision — ElastiCache in front of Aurora, specifically for this service — Why Product gets a cache layer that Order doesn't

Billing Management — extracting invoicing without duplicating Order's data

01. Overview

Billing Management owns invoices, payment records, and ledger entries. It references orders by `order_id` and stores no copy of any order's line items.

Traffic is spiky, concentrated hard around reconciliation and month-end close.

02. Data ownership

Zero dedicated tables exist for Billing today. Payment (`tbl_payment`) and Invoice (`tbl_invoice`) currently live inside Order's package.

Standing up Billing means extracting two existing tables out of Order's package, plus adding a `ledger_entries` table that doesn't exist yet.

Why the migration has to be physical: the tables must move to Billing's own schema and cluster so that no query can join Billing data against Order's tables directly — see [Order Management §02](#).

03. Communication

Almost entirely reactive: one async inbound stream from Order, one on-demand sync call back to Order, one async outbound stream to Notification.

Async IN Order Management → Billing

`OrderCreated` / `OrderStatusChanged` via `EventBridge`

- Billing can lag behind Order by seconds without the buyer noticing.

Sync OUT Billing → Order Management

line-item lookup, at invoice-render time only

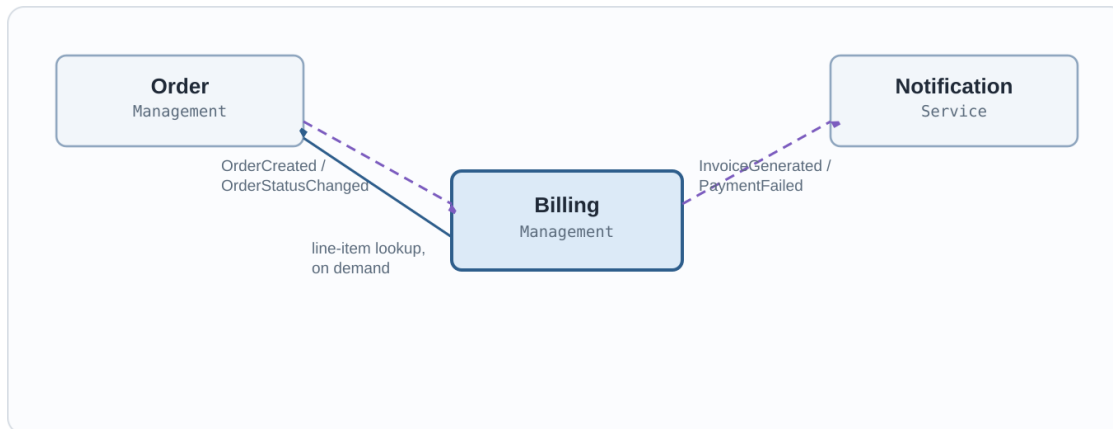
- Used to render the invoice and then discarded — never persisted.

Async OUT Billing → Notification Service

`InvoiceGenerated` / `PaymentFailed` events

- Fire-and-forget, consistent with the platform-wide rule.

04. Architecture diagram



Legend: sync (blocking) async (event)

05. Deployment

ECS Fargate with bursty autoscaling for month-end spikes, Aurora PostgreSQL physically separate from Order's.

modules/billing-management/main.tf

```
// deliberately its own cluster – enforces ADR-004 at the infra level
resource "aws_rds_cluster" "billing_db" {
  cluster_identifier      = "billing-mgmt-${var.environment}"
  engine                  = "aurora-postgresql"
  backup_retention_period = 14
}

module "billing_ecs_service" {
  source          = "../ecs-fargate-service"
  service_name    = "billing-management"
  desired_count   = var.task_count_floor
  max_count       = var.task_count_burst_ceiling
  autoscale_on    = "request_count"
}
```

06. Scaling & region topology

Single-writer with warm standby, per [ADR-007](#), with autoscale burst capacity layered on top for month-end volume.

07. Decisions worth writing down

Decision — physically separate database, never a shared schema — Billing's Aurora cluster is its own resource, distinct from Order's

Makes cross-boundary joins impossible at the infrastructure level — the direct implementation of [ADR-004](#).

Decision — line-item detail is fetched at render time, never persisted — Sync call with a short-TTL cache, not a stored copy

Accept a runtime dependency on Order being reachable, in exchange for zero duplicated data.

08 · Domain service

Notification Service — async-only by construction, not convention

The smallest service in the system by data footprint, and the one with the strictest architectural rule: nothing is allowed to call it synchronously, and the deployment shape makes that structurally true.

01. Overview

Notification Service holds no persistent business data — at most a delivery log. Fully event-driven end to end, scales to zero between bursts.

One rule governs everything: **it must never be called synchronously by another domain service**. Notification delivery depends on third-party providers (SES, SNS) whose latency is outside the platform's control.

02. Data ownership

Zero entities or tables today. 100% greenfield.

Proposal, not existing fact — a delivery log, if added, should be a single append-only table:

proposed: notification_log

```
-- append-only no updates, no foreign keys into other domains
CREATE TABLE notification_log (
  id          uuid PRIMARY KEY,
  recipient   text,
  channel     text,
  template    text,
  status      text,
  sent_at     timestampz
)
```

03. Communication

Every inbound call is asynchronous. There is no synchronous inbound path, by design.

Async IN Order Management

order confirmation

- Order's own request completes without waiting on delivery.

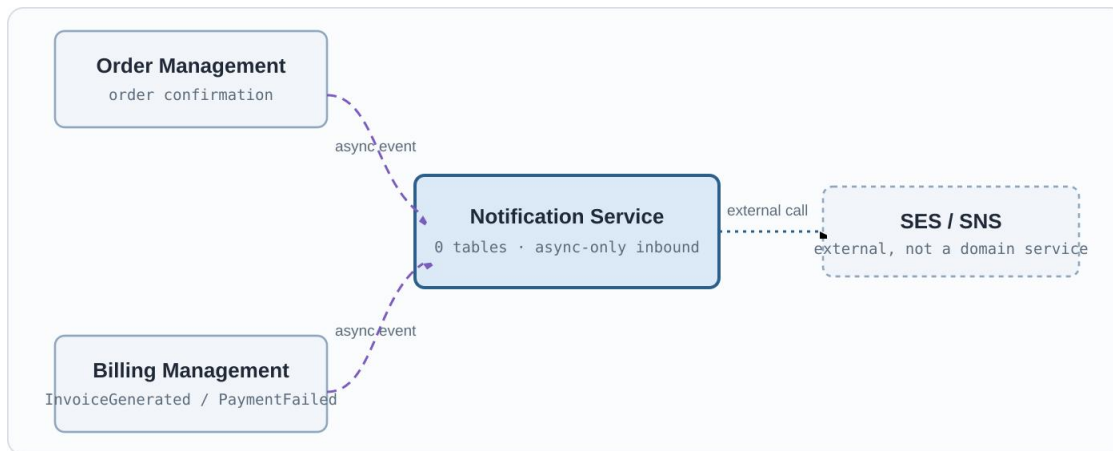
Async IN Billing Management

InvoiceGenerated / PaymentFailed

- Billing fires the event and moves on.

Why zero sync inbound calls, by design: an order confirmation shouldn't fail because an email provider is having a bad five minutes.

04. Architecture diagram



05. Deployment

AWS Lambda, not Fargate — purely event-driven and spiky, scales to zero between bursts.

modules/notification-service/main.tf

```
// event-driven compute: no standing task, no idle cost
resource "aws_lambda_function" "notification_handler" {
  function_name = "notification-handler-${var.region_short}"
  memory_size   = 512
  timeout       = 30
}

resource "aws_cloudwatch_event_rule" "notification_triggers" {
  event_pattern = jsonencode({ "detail-type" = ["OrderConfirmed", "InvoiceGenerated", "PaymentFailed"] })
}

resource "aws_ses_email_identity" "sender" { email = var.notification_from_address }
resource "aws_sns_topic" "sms_outbound" { name = "notification-sms-${var.region_short}" }
```

06. Scaling & region topology

The simplest region story in the whole system — no persistent writer, nothing to fail over, no data migration step.

07. Decisions worth writing down

Decision — Lambda over Fargate — Idle cost doesn't pay for itself on a spiky, event-driven workload

Mirrors the same reasoning applied to Search's index-build consumer.

Decision — no REST endpoint exposed to internal callers — "Never synchronous" is enforced structurally, not by convention

There is no endpoint to call — a fact about deployment topology, not a code-review checklist item.

Appendix C (continued) — Platform Service Design Documents

09 · Platform service

API Gateway — the platform's sole client entry point

Why every client request, and only client requests, pass through one component.

01. Overview

API Gateway is the platform edge: routing, authN/authZ enforcement, and rate limiting all live here, in front of every domain service.

The Gateway scales **1:1 with total client traffic** across the whole platform. No domain service is directly reachable from the internet.

02. Data ownership

None. API Gateway holds no persistent business data and no config store of its own — routing rules and auth policy are configuration it reads from Config Service and the identity provider.

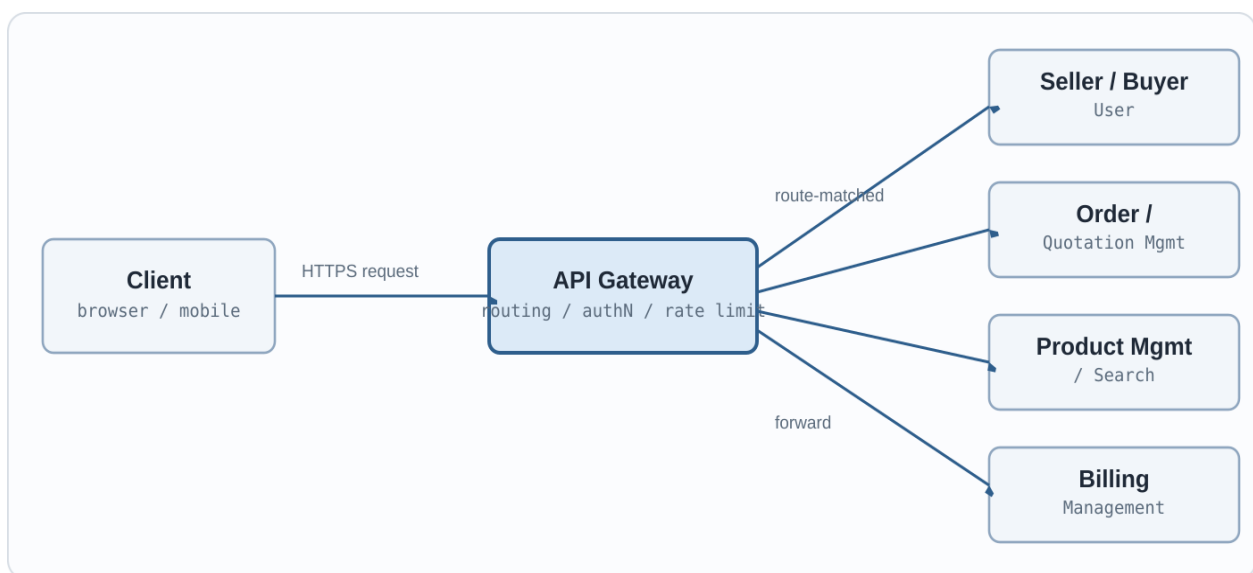
03. Communication

Sync IN — from every client for all reads and writes across the platform.

Sync OUT — to whichever domain service the matched route maps to.

Explicitly out of scope: service-to-service calls. Those bypass the Gateway entirely and resolve directly via Cloud Map / [Discovery Service](#).

04. Architecture diagram



Legend: sync (blocking) — the only edge type here

05. Deployment

Amazon API Gateway (managed) is the primary choice — routing, throttling, Cognito/JWT authorizer, no cluster to run.

Fallback: self-hosted **Spring Cloud Gateway on ECS Fargate** — an explicit escape hatch, not a plan.

modules/api-gateway/main.tf

```
// managed HTTP API – routes fan out to domain services over private integrations
resource "aws_apigatewayv2_api" "platform" { protocol_type = "HTTP" }

resource "aws_apigatewayv2_authorizer" "jwt" {
  api_id           = aws_apigatewayv2_api.platform.id
  authorizer_type  = "JWT"
  jwt_configuration { audience = [var.cognito_app_client_id] issuer = "https://cognito-idp.${var.aws_region}.amazonaws.com/${var.cognito_user_pool_id}" }
}

resource "aws_apigatewayv2_route" "orders" {
  api_id           = aws_apigatewayv2_api.platform.id
  route_key       = "ANY /orders/{proxy+}"
  authorization_type = "JWT"
  authorizer_id    = aws_apigatewayv2_authorizer.jwt.id
}
```

06. Scaling & region topology

Recommendation: one gateway per region, with Route 53 latency-based routing in front — matches [ADR-006](#)'s one-click-per-region rollout goal.

07. Decisions worth writing down

Decision — all client traffic through the Gateway, zero direct exposure — No domain service is directly reachable from the internet

See [ADR-008](#).

Decision — managed by default, self-hosted only on concrete need — Amazon API Gateway first Spring Cloud Gateway is an escape hatch, not a plan

Self-hosted stays on the shelf until a specific requirement the managed offering can't express is identified.

10 · Platform service

Config Service — externalized config, high availability over throughput

Per-service, per-environment, per-region configuration, pulled at boot and refreshed live without a redeploy.

01. Overview

Config Service externalizes configuration out of application code and deploy artifacts. Traffic is low, but it sits on the critical path at exactly the moments that matter — every service's boot blocks on it. The design target is **high availability, not high throughput**.

02. Data ownership

No business tables. Owns **configuration key/value state** — AppConfig hierarchy, pointing to Parameter Store for plain values.

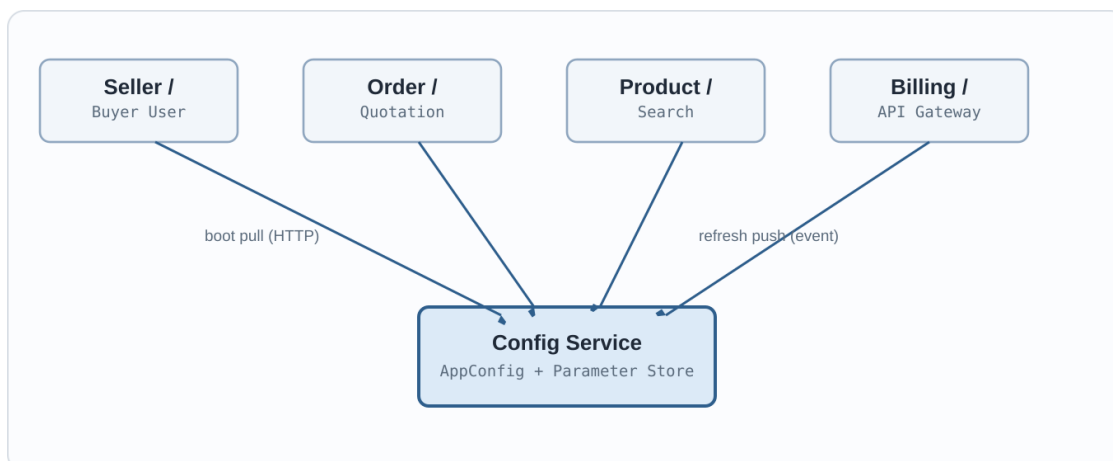
Secrets are explicitly out of this store — pulled from Secrets Manager instead, a real security boundary.

03. Communication

Sync IN — from every service, in two patterns: a boot pull (HTTP), and a refresh event (webhook), not polling.

The refresh-event model is what makes the one-click region rollout in [ADR-006](#) actually work without touching running services.

04. Architecture diagram



Legend: sync (blocking) — boot pull refresh webhook triggers the same pull path

05. Deployment

Option	Choice	Operational cost	Why / why not
A	managed AWS AppConfig + Parameter Store	No cluster to run	Chosen. No new deployment target — set of AWS API calls the service SDK already knows, per ADR-002 .
B	self-hosted Spring Cloud Config Server, Git-backed	A stateful service to deploy, scale, patch per region	Adds a whole new component purely to duplicate a managed service.

modules/config-service/main.tf

```
// one AppConfig application per platform, one environment per deploy target
resource "aws_appconfig_application" "platform" { name = "pharma-aggregator"
}

resource "aws_appconfig_configuration_profile" "order_mgmt" {
  application_id = aws_appconfig_application.platform.id
  name          = "order-management"
  location_uri  = "ssm-parameter://${aws_ssm_parameter.order_mgmt_config.name}"
}
```

06. Scaling & region topology

Provisioned identically per region via the same Terraform module, matching [ADR-006](#). Availability matters far more than latency here.

07. Decisions worth writing down

Decision — AWS-native over self-hosted Config Server — AppConfig + Parameter Store, no cluster to operate per region

See [ADR-002](#).

Decision — secrets never live in Config Service's own store — Secrets Manager is the only place secrets are read from

A compromised Config Service read path can leak configuration, but never credentials.

11 · Platform service

Discovery Service — the live registry that wires regions together

Who's up, in which region, at which address — the registry every service registers with and queries.

01. Overview

Discovery Service is a live service registry: who's up, in which region, at which address, right now. Every service registers on boot and queries it to resolve peers.

02. Data ownership

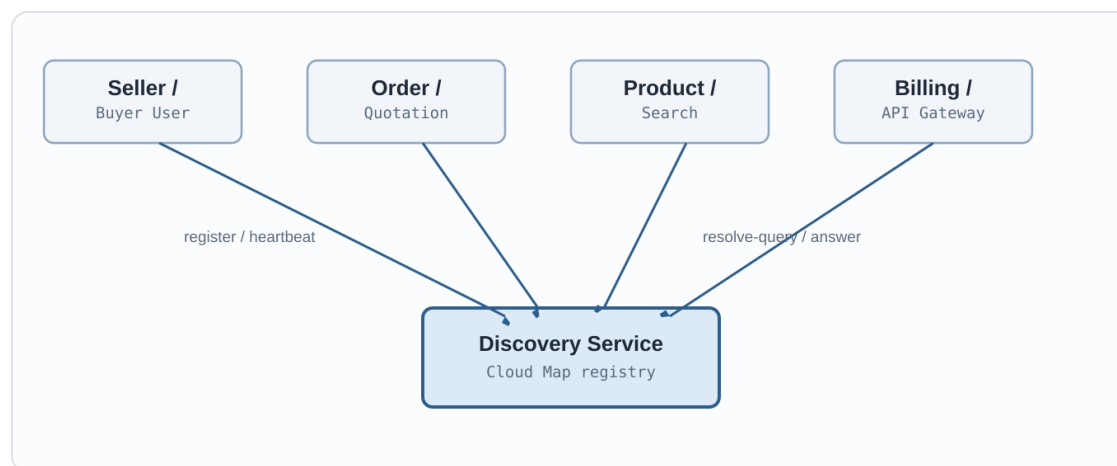
No business tables. Owns **live service-instance registration state** — Cloud Map namespace/service/instance records — **ephemeral and health-checked**, not a durable database.

03. Communication

Platform IN, from every service: registration on boot, deregistration on shutdown, continuous health-check heartbeat.

Every service also queries Discovery to resolve peers for direct service-to-service calls — the canonical example is Order resolving Product's address.

04. Architecture diagram



Legend: sync (blocking) — registration, heartbeat, resolve-query

05. Deployment

Option	Choice	Operational cost	Why / why not
A	managed AWS Cloud	No cluster to run	Chosen. Integrates directly with ECS service discovery, per ADR-002 .

Option	Choice	Operational cost	Why / why not
	Map (or ECS Service Connect)		
B	self-hosted Netflix Eureka or HashiCorp Consul	A stateful HA cluster to run and patch, per region	Duplicates what ECS already speaks to natively.

modules/discovery-service/main.tf

```
// one private DNS namespace per region every service registers its instances
into it
resource "aws_service_discovery_private_dns_namespace" "region" {
  name = "internal.${var.region_short}.pharma-aggregator"
  vpc   = var.vpc_id
}

resource "aws_service_discovery_service" "order_management" {
  name = "order-management"
  dns_config { namespace_id = aws_service_discovery_private_dns_namespace.region.id routing_policy = "MULTIVALUE" dns_records { type = "A" ttl = 10 } }
  health_check_custom_config { failure_threshold = 1 }
}
```

06. Scaling & region topology

Provisioned per region via the same Terraform module, matching [ADR-006](#). This is the mechanism that makes region rollout one-click.

07. Decisions worth writing down

Decision — Cloud Map over self-hosted Eureka/Consul — No stateful HA cluster to operate per region

See [ADR-002](#).

Decision — no hardcoded endpoints anywhere — Every peer address is resolved via Cloud Map DNS, never configured by hand

The actual mechanism, not just a principle, behind zero-manual-wiring region rollout.